

UrbanEye

AI-Smart Complaint and Civic Issue Resolution System

(ES-452: Major Project - Dissertation)

Submitted in partial fulfillment of the requirement for the award of the
degree of

Bachelor of Technology

in

Information Technology / Computer Science

Prepared by

Alok Ranjan

Enrollment No.: _____

Under the supervision of

Assistant Professor

Department of Information Technology / Computer Science

Institute Name: _____

May 2026

DECLARATION

I hereby declare that the dissertation titled “**UrbanEye: AI-Smart Complaint and Civic Issue Resolution System**” is the result of my own work carried out for the Major Project course. This work has been prepared in accordance with the academic guidelines prescribed for the degree program. I further declare that this report has not been submitted, either in full or in part, to any other university or institution for the award of any degree or diploma. Wherever the ideas, findings, or words of other researchers have been used, due acknowledgement has been made in the appropriate sections of the report.

Alok Ranjan

Student

Date: _____

CERTIFICATE

This is to certify that the dissertation work titled “**UrbanEye: AI-Smart Complaint and Civic Issue Resolution System**” has been carried out by **Alok Ranjan** under my supervision and guidance in partial fulfillment of the requirements for the award of the Bachelor of Technology degree. To the best of my knowledge, the work presented in this report is original and has not been submitted elsewhere for any other degree or diploma.

Project Supervisor

Head of Department

ACKNOWLEDGEMENT

The completion of this project has been possible only because of the support, encouragement, and guidance I received from many people. I would like to express my sincere gratitude to my project supervisor for continuous guidance, valuable suggestions, and constructive criticism at every stage of the work. Their insight helped convert a simple application idea into a structured system that addresses a real civic problem.

I am equally thankful to the faculty members of the department for providing the academic environment and technical foundation required to carry out this project. Their classes and discussions contributed significantly to my understanding of software engineering, full-stack development, and intelligent system design.

I would also like to acknowledge my friends and peers who helped with testing, feedback, and discussion during the design and implementation phases. Their practical observations improved the usability of the web and mobile interfaces and helped identify gaps in the complaint workflow.

Finally, I would like to thank my family for their constant encouragement and patience throughout the project. Their support gave me the motivation to complete the work with sincerity and confidence.

ABSTRACT

UrbanEye is an AI-assisted civic issue reporting and complaint resolution platform designed to improve the way citizens interact with local civic administration. Traditional complaint workflows are usually fragmented, slow, and poorly tracked. Citizens often register issues through separate channels, receive little visibility into progress, and rarely get structured public communication once a complaint is escalated. UrbanEye addresses this challenge through a unified digital system that supports complaint reporting, categorization, routing, status monitoring, and AI-generated public update content.

The system combines a web application for dashboard access, a mobile application for field-friendly reporting and tracking, a backend API for business logic and persistence, and a FastAPI-based AI service for complaint analysis. When a complaint is filed, the platform captures the title, description, location, and optional image. The AI layer performs rule-based triage with support for language-model-based enhancement, assigns category and priority, suggests the responsible department, and prepares an X-ready social update for official communication workflows.

The platform supports multiple user-facing views, including complaint dashboards, map hotspots, complaint details, administrative statistics, and profile management. The mobile application additionally provides an improved status-centric interface in which residents can quickly determine whether their complaints are pending, in progress, or resolved. A lightweight persistence strategy is used to support both online and fallback operation, allowing the platform to remain demonstrable even when external services are unavailable.

UrbanEye has been developed using Express.js, Prisma, PostgreSQL-ready schemas, FastAPI, React Native with Expo, and Next.js. The project demonstrates how full-stack application engineering and practical AI augmentation can be combined to solve a real-world civic communication and service-tracking problem in a simple, scalable, and user-friendly manner.

LIST OF FIGURES

Figure No.	Figure Title
3.4.1	Use Case Diagram of UrbanEye
3.4.2	Level 0 Data Flow Diagram
3.4.3	Level 1 Complaint Processing DFD
4.1.1	Work Breakdown Structure
4.2.1	System Architecture Diagram
4.2.2	Module Interaction Diagram
4.3.1	Activity Diagram for Complaint Submission
4.3.2	Activity Diagram for AI Analysis and Status Tracking
4.4.1	ER Diagram
4.4.2	Class / Data Model Diagram
5.1.1	Web Home and Dashboard Flow
5.1.2	Mobile Home and Dashboard Flow

LIST OF TABLES

Table No.	Table Title
1.1	Major Objectives of UrbanEye
2.1	Problems in Conventional Civic Complaint Workflows
3.1	Functional Requirements
3.2	Non-Functional Requirements
3.3	Tools, Technologies, and Platforms Used
4.1	Core Modules of the Proposed System
6.1	Testing Summary

TABLE OF CONTENTS

Declaration	I
Certificate	II
Acknowledgement	III
Abstract	IV
List of Figures and List of Tables	V
Chapter 1: Introduction	1
Chapter 2: Problem Statement	5
Chapter 3: Analysis	9
Chapter 4: Design and Architecture	17
Chapter 5: Implementation	31
Chapter 6: Testing	42
Chapter 7: Summary and Conclusion	47
Chapter 8: Limitations and Future Work	50
Bibliography	53
Appendix	54

CHAPTER 1: INTRODUCTION

1.1 Project Overview

UrbanEye is a full-stack civic complaint management platform that brings together complaint reporting, routing, analytics, AI-assisted triage, and user-facing status tracking in a single system. The core idea behind the project is simple: residents should be able to report a civic problem in one place, receive meaningful visibility into what happens next, and trust that the complaint is being directed to the correct department. At the same time, authorities need a lightweight operational view of complaint trends, priorities, and hotspots so that action can be better coordinated.

The project was developed as a multi-surface application instead of a single website because civic complaint handling naturally involves different stakeholders and environments. The web application supports dashboard-driven review and broader oversight, the mobile application supports quick complaint entry and resident-side tracking, the Express backend provides a structured API and persistence layer, and the FastAPI AI service produces triage intelligence and social communication text. This split makes the solution modular while keeping the user workflow coherent.

1.2 Need for the Project

In ordinary civic workflows, complaint handling suffers from four recurring issues: fragmented reporting channels, weak classification, poor transparency, and delayed communication. A resident may know that an issue such as garbage overflow, streetlight failure, drainage blockage, or water leakage exists, but may not know which department is responsible or whether the complaint has moved beyond initial registration. In many cases, the complaint is acknowledged but not clearly tracked from the citizen's perspective.

UrbanEye was designed to reduce that uncertainty. By classifying complaints into categories such as sanitation, electricity, water, and roads, the system supports better routing. By maintaining complaint state fields such as pending, in progress, and resolved, the platform gives the resident a more interpretable progress trail. By producing structured dashboards and hotspot views, it also helps turn individual complaints into actionable operational visibility.

1.3 Major Objectives

Objective	Description
Unified Complaint Lifecycle	Provide reporting, analysis, tracking, and public update preparation in one connected system.

AI-Assisted Triage	Automatically infer category, priority, department, suggested action, and social communication text.
Multi-Platform Access	Support both web and mobile access so that residents and operators can use the system conveniently.
Administrative Visibility	Present complaint counts, status breakdowns, and map hotspots for monitoring and follow-up.
Status Transparency	Allow residents to clearly see whether their complaint is pending, being handled, or solved.
Extensibility	Keep the architecture open for future integration with real social publishing, maps, and department workflows.

1.4 Scope of the Project

The scope of UrbanEye includes user registration and login, complaint submission with text and optional image, AI-assisted complaint analysis, dashboard and hotspot monitoring, mobile complaint tracking, and basic administrative analytics. The system is intended as a robust academic prototype with production-ready patterns in several modules, especially in API structure, data modeling, and fallback handling.

The present scope does not yet include field workforce dispatch automation, actual X publishing credentials, GIS mapping integration beyond a simplified hotspot representation, or a complete role-based workflow engine for department authorities. However, the current architecture is designed in a way that such features can be introduced incrementally.

Files in project 87,935	Code lines 358,025 total	Platforms Web, Mobile, Backend, AI
-----------------------------------	------------------------------------	--

CHAPTER 2: PROBLEM STATEMENT

2.1 Problem Definition

Civic service complaint systems often fail not because people are unwilling to report issues, but because the path from complaint to resolution is unclear. A user may file a complaint and then lose sight of what happens next. The responsible department may not be obvious, complaints may not be categorized consistently, and administrators may not have an immediate cross-channel view of the workload. This creates dissatisfaction for residents and inefficiency for authorities.

The problem becomes more serious when manual handling is involved. Without structured category labels, a water complaint may be mixed with road maintenance tasks. Without a priority signal, urgent incidents may be delayed. Without a public communication layer, residents may assume inaction even when internal processing has started. Thus, the core problem is both operational and communicative.

2.2 Existing System Drawbacks

Issue	Description	Impact
Fragmented Reporting	Residents use different channels or ad hoc mechanisms to submit civic issues.	Records become inconsistent and hard to track.
Weak Classification	Complaints are often stored as plain text without structured category or priority labels.	Wrong routing and delayed action.
Poor Status Visibility	Users are not clearly informed whether the issue is open, under action, or resolved.	Low trust and repeated follow-up requests.
No Communication Support	Public-facing updates require manual drafting every time.	Slow information flow and inconsistent messaging.
Limited Administrative Insight	Authorities cannot easily view hotspots, departmental breakdowns, or recent patterns.	Reactive instead of proactive decision-making.

2.3 Proposed Solution

UrbanEye proposes a layered digital system in which complaints move through a structured lifecycle. The user submits a complaint through web or mobile. The backend forwards the essential textual content to an AI service that performs local classification and can optionally enhance output using an LLM. The enriched complaint, along with its department, priority, suggested action, and X-ready

social update, is then stored or returned through the backend. The same complaint becomes visible in user dashboards, hotspot summaries, and administrative views.

Instead of treating AI as a separate showcase feature, the project uses it to reduce practical friction in the complaint-routing workflow. The AI component is therefore directly tied to utility: it shortens triage time, standardizes categorization, and supports communication readiness.

2.4 Specific Objectives

1. To build a complaint registration interface for citizens on both web and mobile platforms.
2. To design a backend API capable of storing, retrieving, and presenting complaint status information.
3. To implement an AI-assisted analysis module for category, priority, department, and social update generation.
4. To provide an administrative dashboard with counts, breakdowns, and complaint hotspot visibility.
5. To improve trust and transparency through resident-facing complaint status tracking.
6. To keep the system modular enough for future integration with maps, notifications, and social publishing.

CHAPTER 3: ANALYSIS

3.1 Software Requirement Specification

3.1.1 Functional Requirements

ID	Requirement	Description
FR-1	User Authentication	The system shall allow residents to register and log in through the backend authentication API.
FR-2	Complaint Submission	The system shall accept title, description, location, and optional image for complaint creation.
FR-3	Complaint Analysis	The AI service shall produce category, priority, department, sentiment, confidence, suggested action, and social post.
FR-4	Status Tracking	The user shall be able to view complaint status as pending, in progress, or resolved.
FR-5	Admin Analytics	The system shall provide aggregate counts and complaint distribution for administrative review.
FR-6	Hotspot Map Data	The system shall expose complaint hotspot information for locality-based visibility.
FR-7	X Account Preference	The mobile application shall allow the user to save a preferred X handle for update readiness.
FR-8	Offline/Fallback Support	The applications shall return usable fallback data when external services are unavailable.

3.1.2 Non-Functional Requirements

Category	Requirement
Usability	Interfaces should be simple enough for residents while still exposing meaningful admin insight.
Performance	Complaint analysis and dashboard loading should remain responsive for ordinary payload sizes.

Maintainability	Frontend, backend, and AI services should remain separable and independently maintainable.
Scalability	The data model and API patterns should support migration from mock storage to PostgreSQL-backed persistence.
Reliability	The system should continue functioning in development or degraded service mode using local fallback logic.
Portability	The product should be accessible through both browser and mobile interfaces.

3.2 Feasibility Study

Technical Feasibility

The project is technically feasible because its components rely on stable and well-documented technologies. Express.js is suitable for API development, Next.js is effective for dashboard-driven web interfaces, Expo supports rapid React Native development, and FastAPI is well suited for lightweight AI microservices. The use of Prisma with a PostgreSQL-ready schema further improves maintainability and future expansion.

Operational Feasibility

The proposed system is operationally feasible because the target workflow is understandable to both citizens and authorities. Residents only need to submit a description and location, while authorities or administrators benefit from pre-computed analytics. The AI component reduces manual interpretation rather than introducing extra complexity.

Economic Feasibility

The development stack used in UrbanEye is cost-effective for an academic and prototype-to-deployment transition. Most components are open source or have student-friendly plans. The architecture can initially run in a demo mode using local/mock data and later be upgraded to managed storage and hosted deployment when necessary.

3.3 Tools / Technologies / Platform Used

Layer	Tools and Technologies	Purpose
Web Frontend	Next.js 16, React 19, TypeScript, Tailwind CSS	Citizen/admin dashboard, complaint views, reporting UI

Mobile Frontend	React Native, Expo, AsyncStorage	Resident complaint reporting and tracking on mobile devices
Backend API	Node.js, Express.js, Prisma, bcrypt	Authentication, complaint APIs, admin APIs, map APIs
AI Service	FastAPI, Pydantic, optional LangChain-based LLM integration	Complaint triage and social post generation
Persistence	Prisma schema, PostgreSQL-ready models, mock store, AsyncStorage	Complaint/user storage and local fallback persistence
Auxiliary Modules	Expo Image Picker, fetch, REST APIs	Image intake, network communication, mobile submission flow

3.4 Use Case Diagrams / Data Flow Diagrams

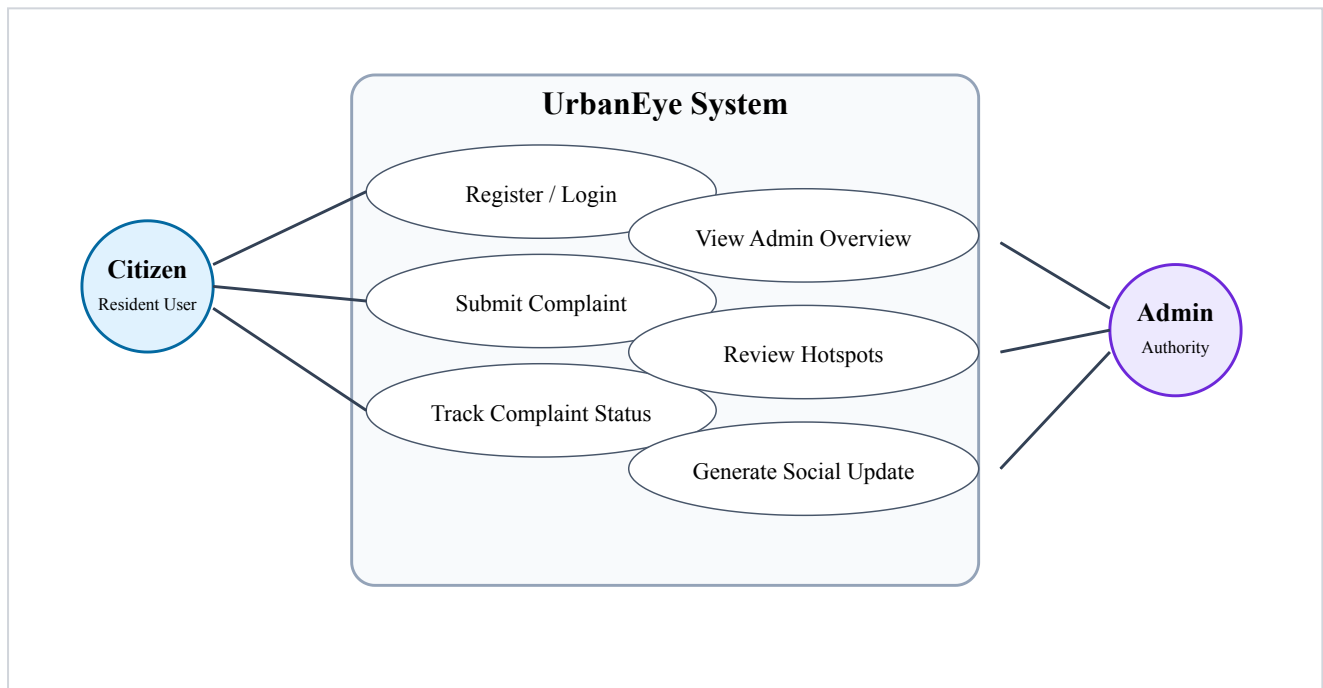


Figure 3.4.1: Use Case Diagram of UrbanEye

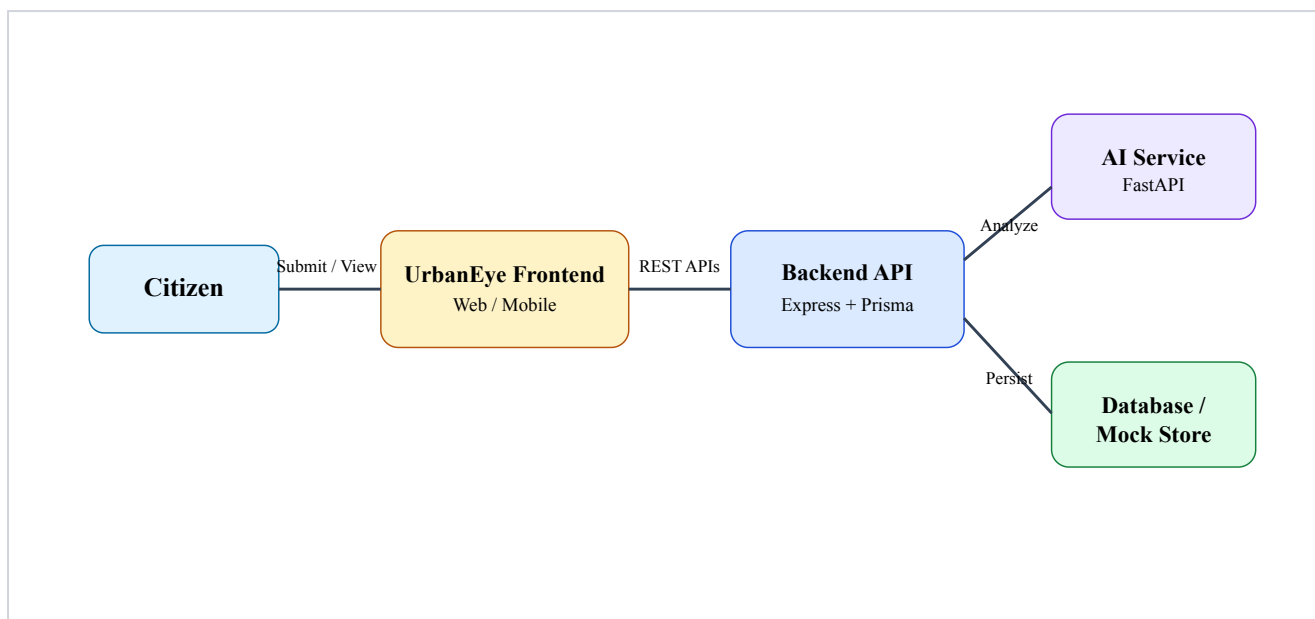


Figure 3.4.2: Level 0 Data Flow Diagram

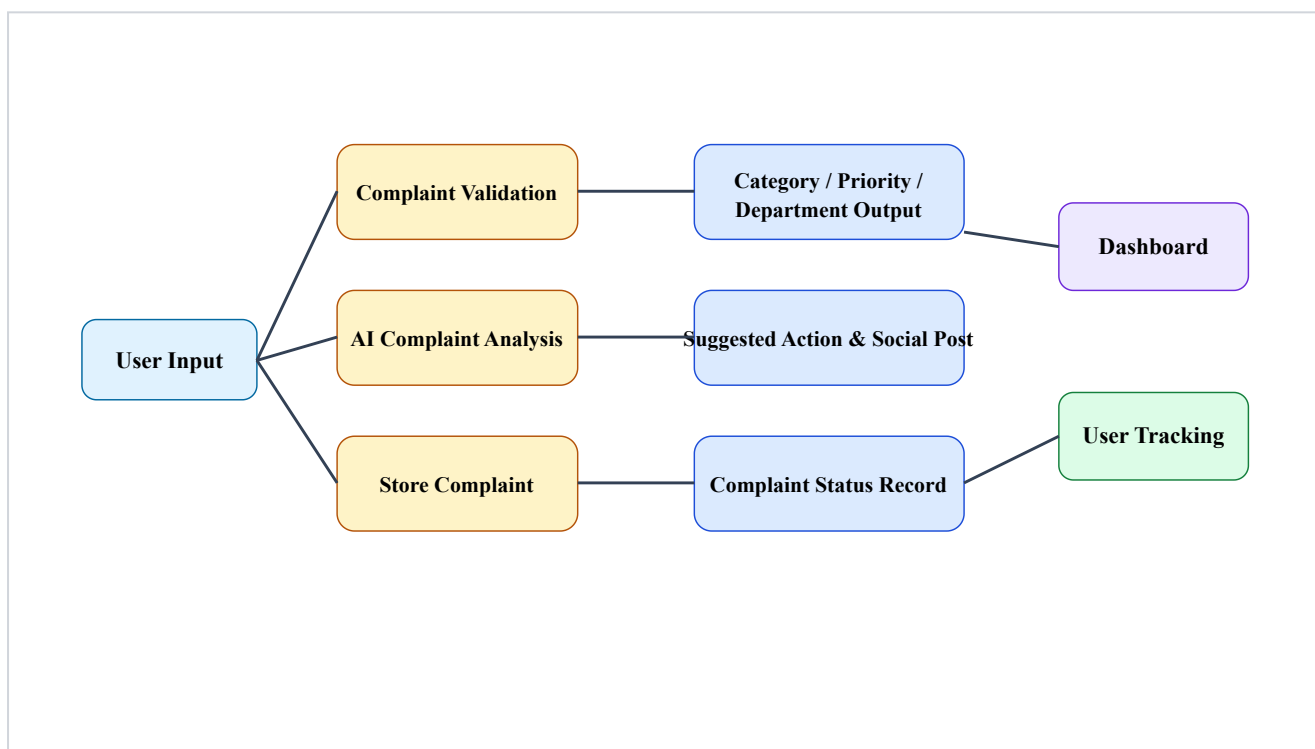


Figure 3.4.3: Level 1 Complaint Processing DFD

CHAPTER 4: DESIGN AND ARCHITECTURE

4.1 Structure Chart / Work Breakdown Structure

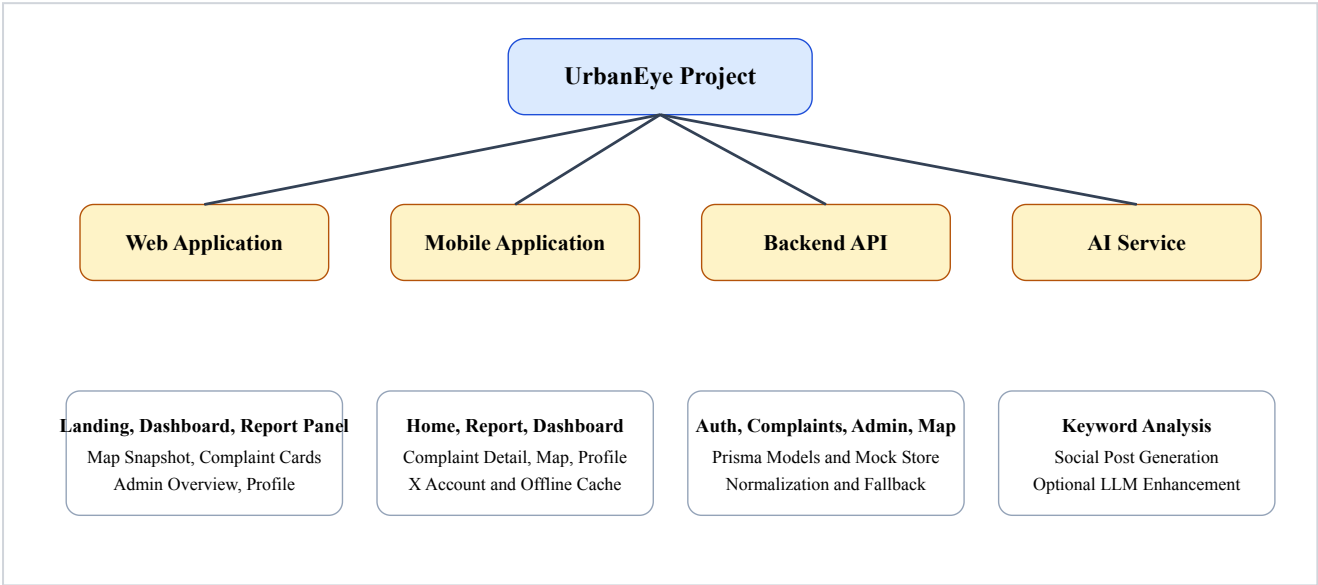


Figure 4.1.1: Work Breakdown Structure

4.2 Explanation of Modules

Module	Description
Authentication Module	Handles resident registration and login through Express routes and bcrypt-protected user records.
Complaint Management Module	Creates complaints, fetches complaint lists and details, and normalizes complaint shape for all clients.
AI Analysis Module	Determines category, priority, department, sentiment, suggested action, and X-ready social post text.
Administrative Dashboard Module	Provides summary counts, priority breakdowns, department distributions, and recent complaint snapshots.
Map Hotspot Module	Exposes complaint latitude/longitude and locality data for hotspot visualization.
Mobile Tracking Module	Shows complaint resolution progress, local persistence, and X handle preferences for residents.

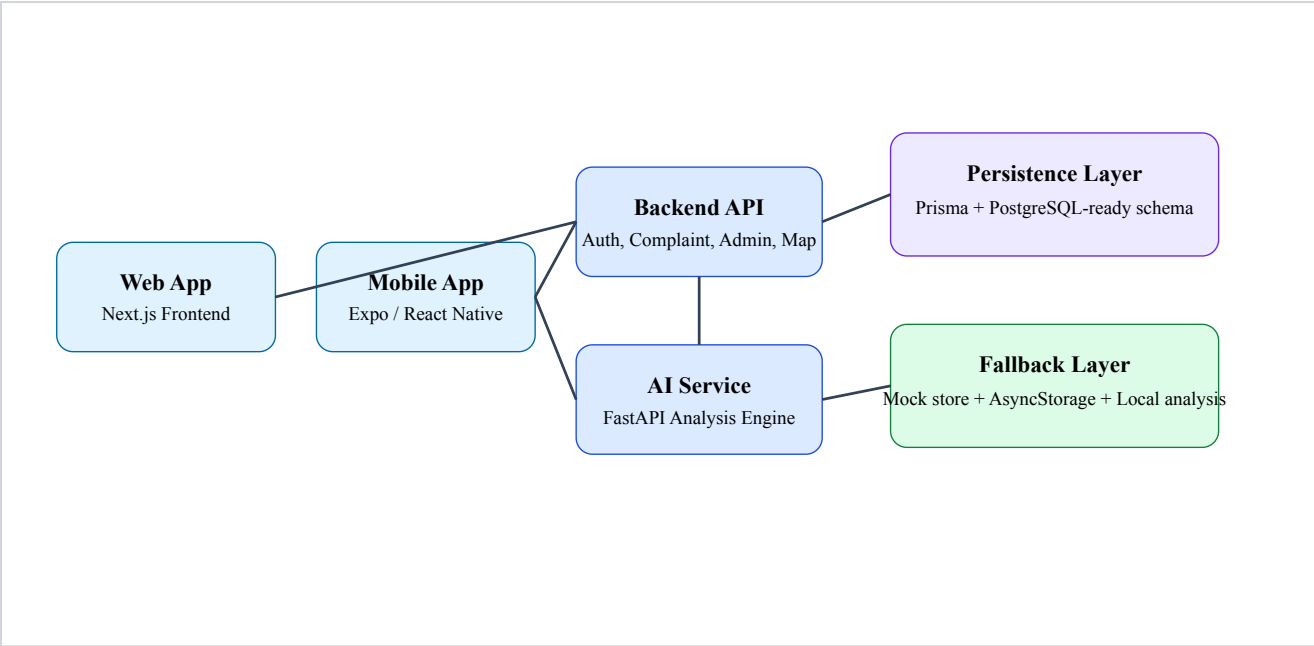


Figure 4.2.1: System Architecture Diagram

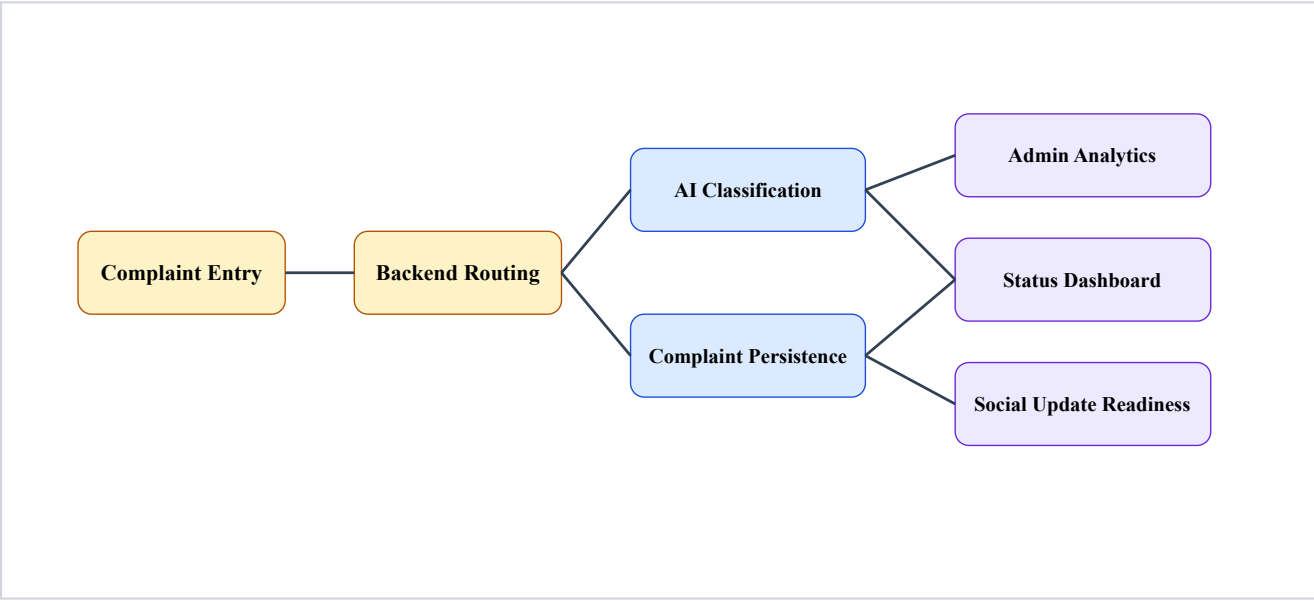


Figure 4.2.2: Module Interaction Diagram

4.3 Flow Chart / Activity Diagram

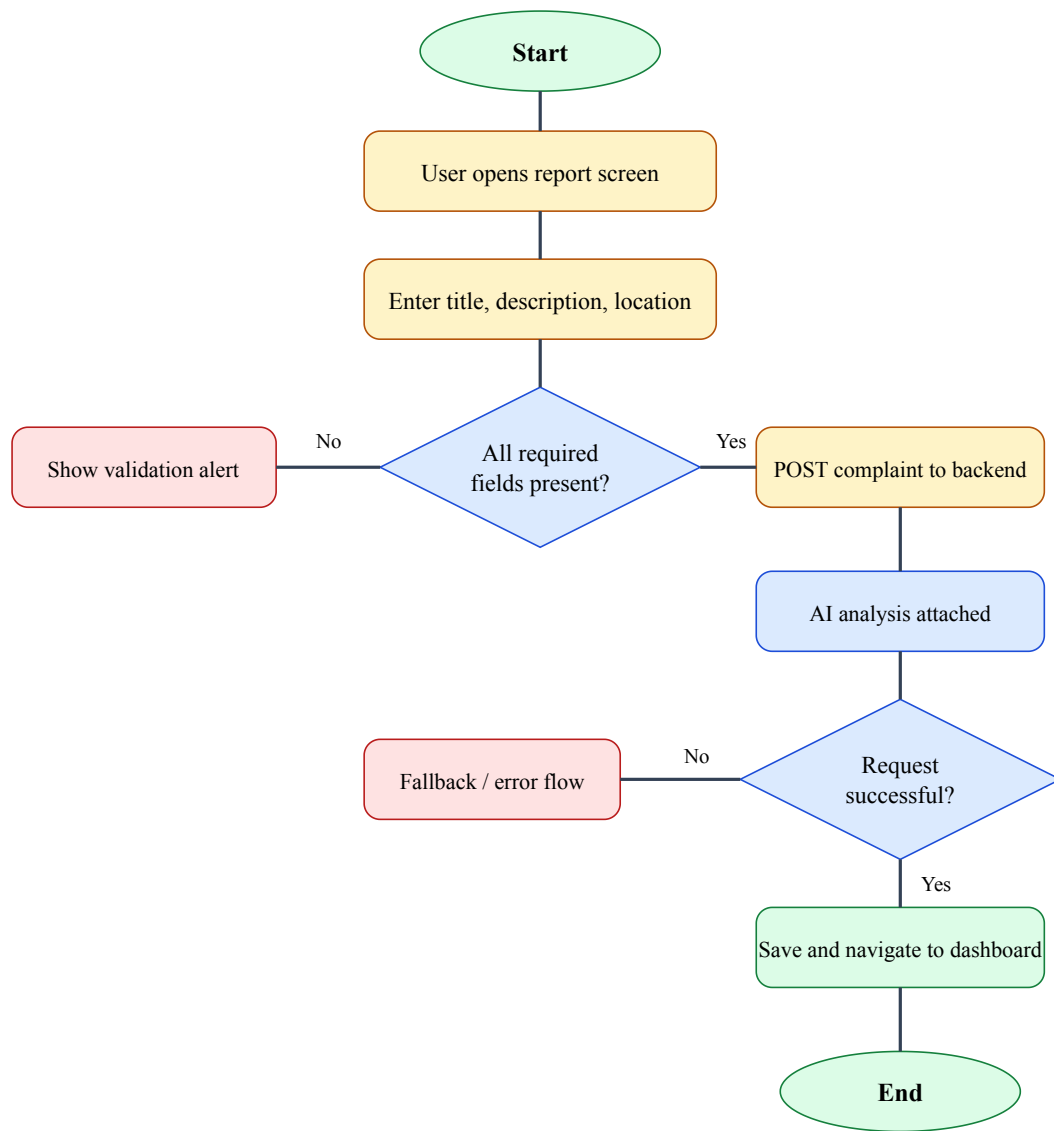


Figure 4.3.1: Activity Diagram for Complaint Submission

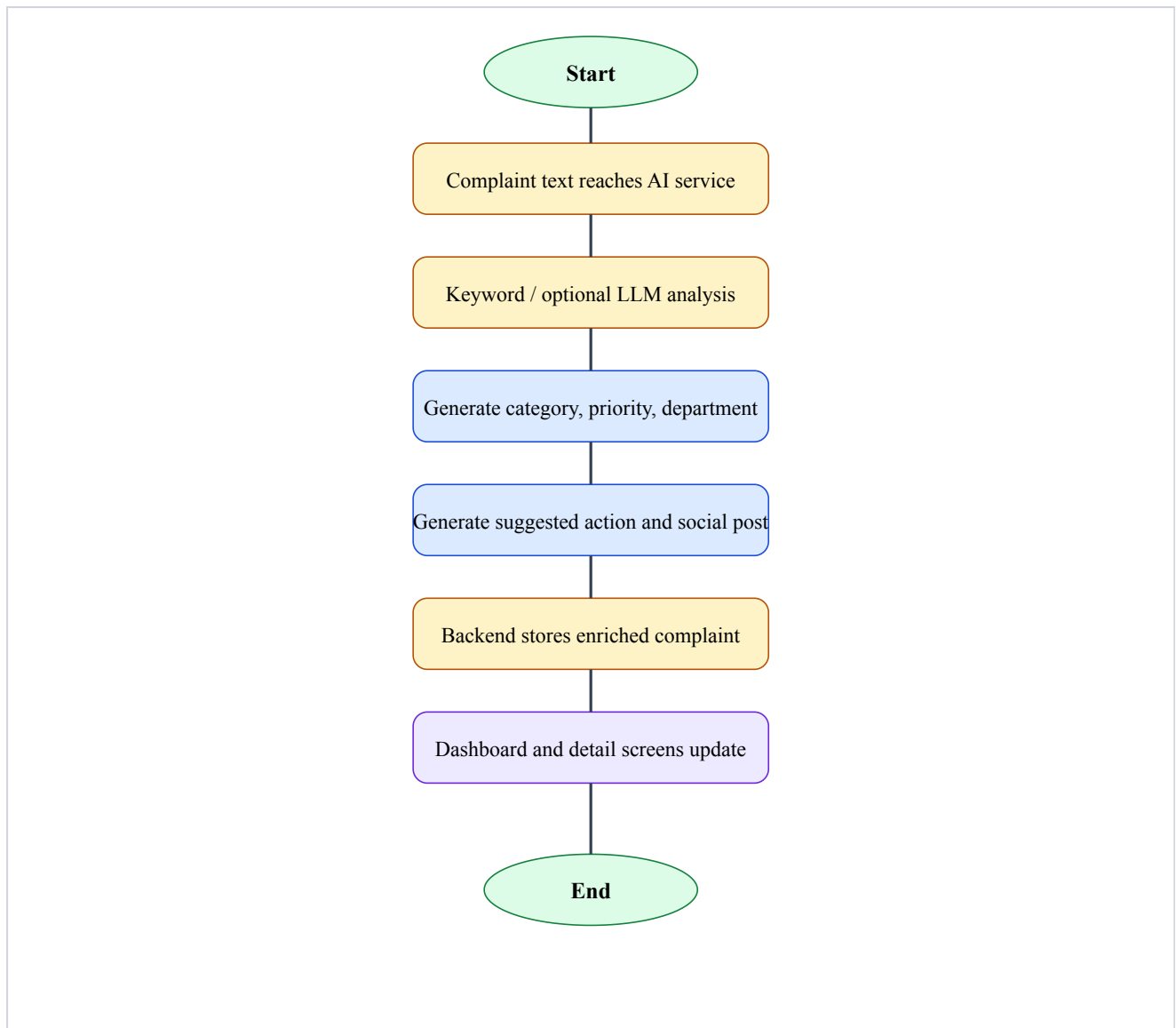


Figure 4.3.2: Activity Diagram for AI Analysis and Status Tracking

4.4 ER Diagram / Class Diagram

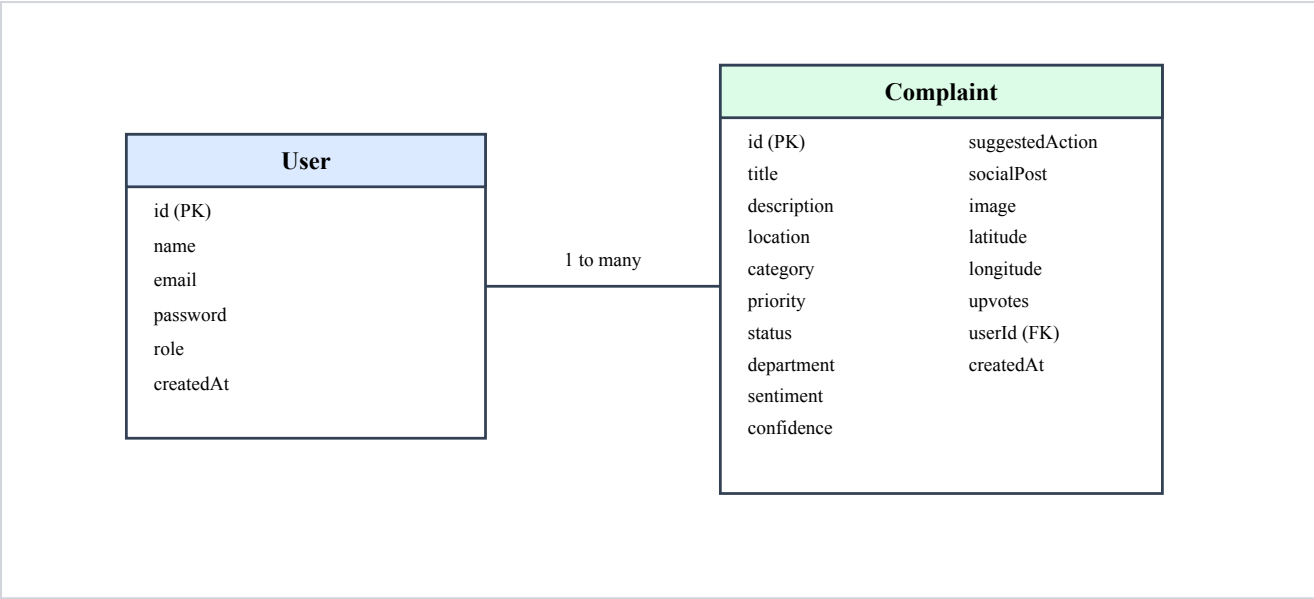


Figure 4.4.1: ER Diagram

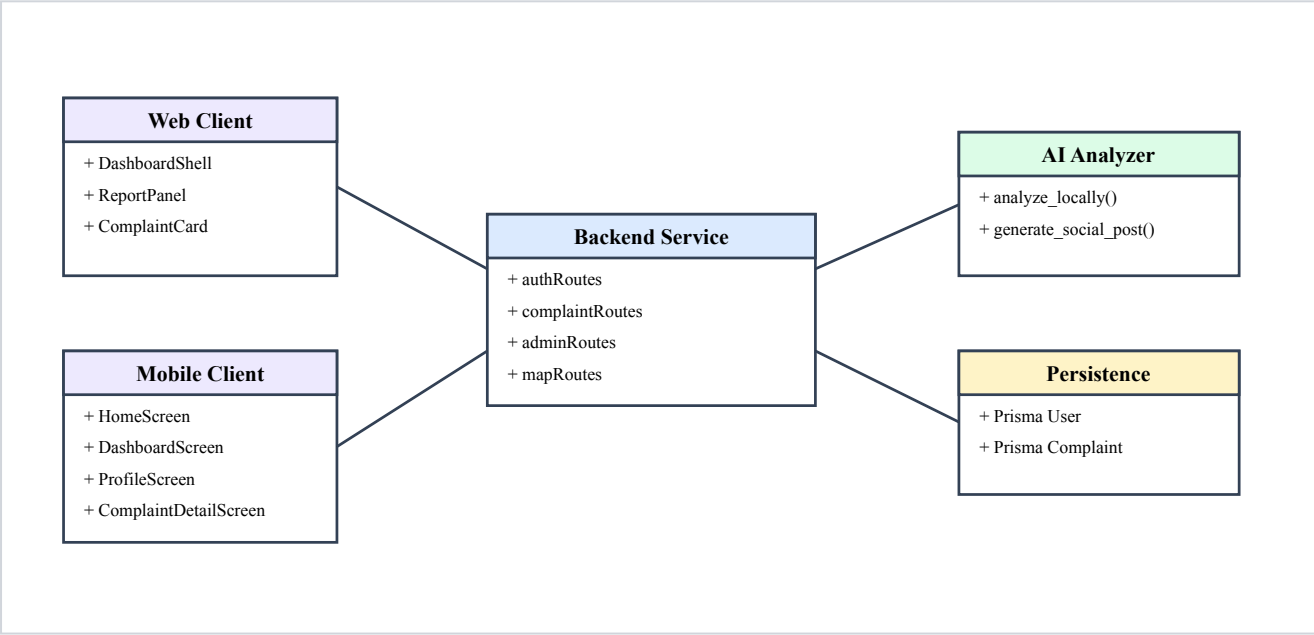


Figure 4.4.2: Class / Data Model Diagram

CHAPTER 5: IMPLEMENTATION

5.1 Application Screens and Working

Web Application

The web application is built in Next.js and functions as the primary dashboard-oriented interface. The landing page presents the civic issue resolution system with complaint snapshots and quick navigation to registration and the operational dashboard. The dashboard combines a reporting panel, complaint cards, hotspot summaries, and administrative overview metrics. This makes the web view suitable for a supervisory or administrative workflow in which multiple complaints must be scanned in a short time.

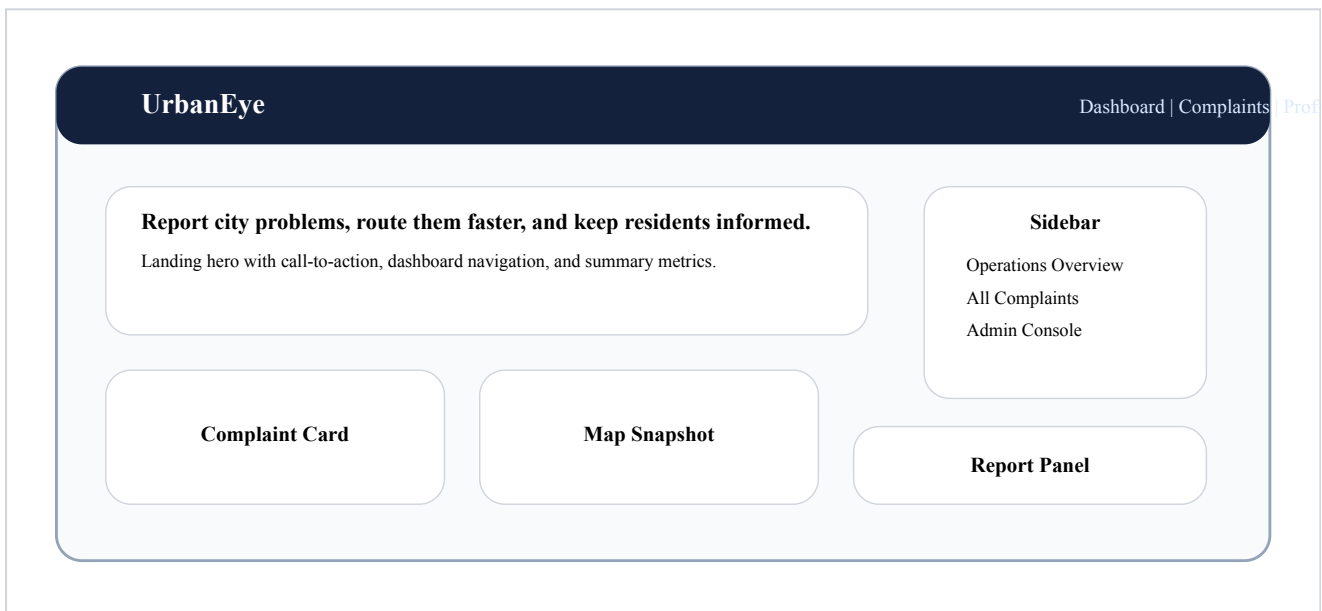


Figure 5.1.1: Web Home and Dashboard Flow

Mobile Application

The mobile application is designed from the citizen's perspective. Its screens focus on quick complaint entry, meaningful status visibility, and reduced friction in follow-up. The home screen now highlights complaint counts and X-account readiness, the dashboard emphasizes the proportion of solved and open complaints, and the profile screen stores a preferred X handle. Local complaint persistence is used so that the dashboard remains informative even under fallback conditions.

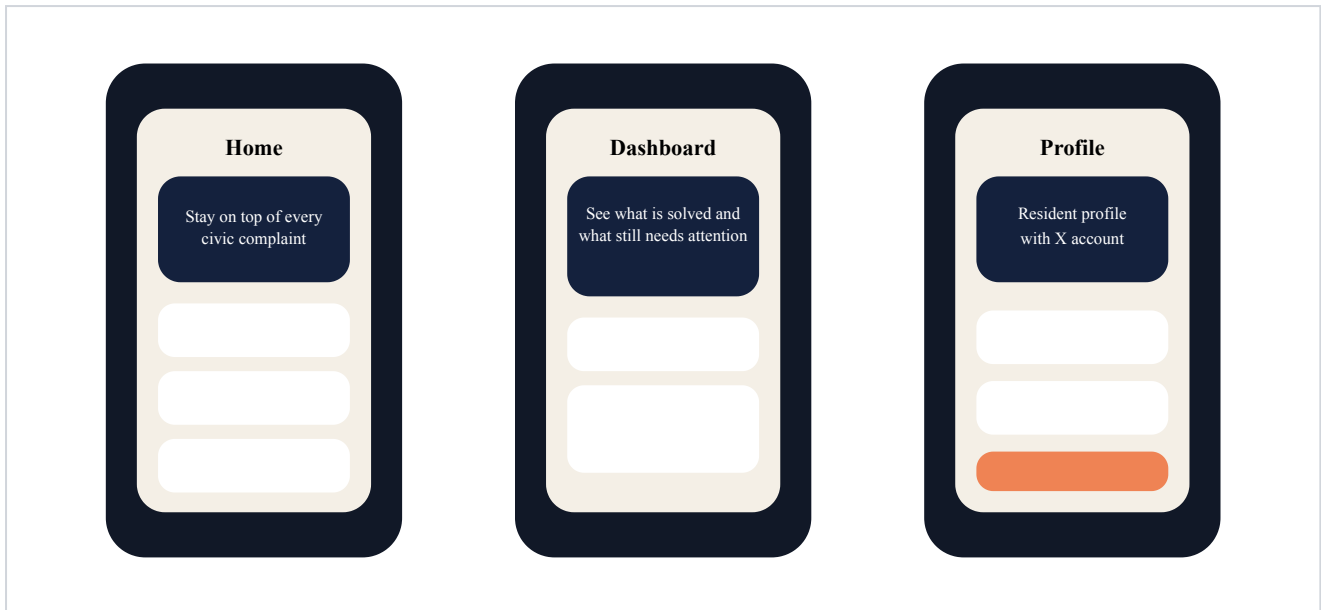


Figure 5.1.2: Mobile Home and Dashboard Flow

5.2 Source Code of Some Modules

Complaint Analysis Flow

The following backend-side AI logic illustrates how a complaint is enriched before being returned to the application. The system relies on keyword-based matching with optional LLM enhancement and then generates a compact social update that can be used for public communication.

```
def analyze_locally(payload: ComplaintPayload) -> AnalysisResult:
    text = f"{payload.title} {payload.description} {payload.location}".lower()

    category = "General"
    department = "Civic Response Cell"
    priority = "MEDIUM"
    sentiment = "concerned"

    if keyword_match(text, ["garbage", "waste", "trash"]):
        category = "Sanitation"
        department = "Sanitation Department"
        priority = "HIGH"
    elif keyword_match(text, ["light", "power", "electric"]):
        category = "Electricity"
        department = "Electricity Department"

    action = f"Route this complaint to {department} and request field validation."
    social = generate_social_post(...)
```

Backend Complaint Creation

```
router.post("/", createComplaint);
router.get("/", getComplaints);
router.get("/:id", getComplaint);

const analysis = await analyzeComplaint({ title, description, location });

const complaint = await prisma.complaint.create({
  data: {
    title: title.trim(),
    description: description.trim(),
    location: location.trim(),
    category: analysis.category,
    priority: analysis.priority,
    department: analysis.department,
    sentiment: analysis.sentiment,
    confidence: analysis.confidence,
    suggestedAction: analysis.suggestedAction,
    socialPost: analysis.socialPost,
  }
});
```

Mobile Complaint Tracking Logic

```
const stats = useMemo(() => getComplaintStats(data), [data]);
const filteredData = useMemo(() => filterComplaints(data, activeFilter),
[activeFilter, data]);
const solvedRate = stats.total ? Math.round((stats.resolved / stats.total) * 100)
: 0;

{DASHBOARD_FILTERS.map((filter) => {
  const isActive = activeFilter === filter.key;
  return (
    <TouchableOpacity key={filter.key} onPress={() =>
setActiveFilter(filter.key)}>
      ...
    </TouchableOpacity>
  );
})}
```

CHAPTER 6: TESTING

6.1 Testing Strategy

The testing strategy for UrbanEye was based on iterative verification of individual modules, API routes, and client interfaces. Since the project is composed of several independently runnable parts, each layer was checked using the most practical method available: syntax checks for source modules, bundling for frontend validation, complaint-flow API calls for backend behavior, and fallback execution for resilience testing.

6.2 Test Cases

Test Case	Input / Action	Expected Result	Observed Result
User Registration	Submit name, email, password	User record created or fallback user returned	Passed
Complaint Submission	Submit title, description, location	Complaint stored with status and analysis metadata	Passed
AI Analysis	Streetlight / garbage / water complaint text	Correct category and department inference	Passed
Social Post Generation	Complaint routed through AI service	X-ready text generated within compact length	Passed
Dashboard Filtering	Select Pending / In Progress / Solved	Complaint list filtered correctly	Passed
Mobile Local Persistence	Create complaint in fallback mode	Complaint remains visible after reload	Passed

6.3 Validation Methods Used

- Python syntax compilation for the FastAPI service modules.
- Node syntax checks for the backend and JavaScript service files.
- Prisma client generation to validate schema consistency.
- Expo Android bundling to ensure the mobile application compiles successfully.
- Next.js development server execution to validate web frontend startup.

6.4 Testing Summary

The application components were found to be functionally coherent for the intended project scope. The backend routes handled both persistent and fallback complaint flows correctly. The AI service generated structured response objects that matched the frontend expectations. The mobile dashboard successfully highlighted solved versus open complaints, while the web dashboard consolidated complaint reporting and oversight. The most notable strength of the test cycle was the presence of fallback mechanisms, which ensured the product remained demonstrable even in a constrained environment.

CHAPTER 7: SUMMARY AND CONCLUSION

7.1 Summary of Work Done

UrbanEye was developed as an integrated civic complaint resolution platform with four major technical layers: web frontend, mobile frontend, backend API, and AI service. The project began with the core requirement of complaint registration and gradually evolved into a more complete civic workflow system that supports complaint analysis, structured routing, resident status tracking, and administrative monitoring.

On the backend side, complaint, authentication, map, and admin routes were organized in Express. The persistence layer was prepared through Prisma models for user and complaint entities, while a mock store ensured that the system remained operable in demonstration mode. On the AI side, a FastAPI microservice was used to generate category, priority, department, sentiment, suggested action, and X-ready update text. The AI flow was deliberately grounded in practical routing utility rather than purely experimental text generation.

The web application presented complaint dashboards and operational summaries, whereas the mobile application focused on a user-friendly citizen journey with complaint reporting, improved status interpretation, and profile-level X account preferences. Together, these components formed a cohesive project that demonstrates the use of full-stack engineering and assistive AI in public-service workflows.

7.2 Conclusion

The dissertation objective of building an AI-assisted civic issue resolution system has been successfully met in the form of UrbanEye. The project demonstrates that even a relatively compact academic system can meaningfully improve complaint handling when structured data, dashboard visibility, and AI-assisted triage are combined. Rather than simply storing complaint text, the system transforms input into actionable and trackable information.

The strongest contribution of the work lies in its workflow integration. Complaint collection, analysis, storage, dashboarding, and communication preparation are not isolated features; they are linked parts of the same service pipeline. This makes UrbanEye a useful foundation for future civic-tech expansion and a strong example of applied software engineering in a public-facing problem domain.

CHAPTER 8: LIMITATIONS AND FUTURE WORK

8.1 Limitations of the Project

1. The current complaint intelligence primarily uses rule-based local analysis, with optional LLM enhancement rather than a fully trained domain model.
2. The complaint status lifecycle is simplified and does not yet include a complete department-side action workflow or staff assignment engine.
3. The map hotspot layer is intentionally lightweight and not yet connected to a full GIS visualization stack.
4. The current system supports X-account preference and X-ready content generation, but not direct authenticated publishing to the X platform.
5. Fallback demo data is still used in some flows to keep the project resilient in local development, which means not every interface always represents real production traffic.
6. The report cover page still requires final institutional details such as supervisor name, department, and enrollment number before university submission.

8.2 Future Scope

1. Integration with official civic department dashboards and authenticated authority accounts.
2. Real-time notifications through email, SMS, and push notification services.
3. Direct X publishing for approved official communication after human confirmation.
4. Geospatial clustering and heatmap-based issue prioritization for city-scale deployments.
5. Machine-learning models trained on historical civic complaints to improve categorization and urgency scoring.
6. Complaint assignment and field staff tracking for resolution accountability.
7. Multilingual complaint intake for wider accessibility.
8. Evidence-based analytics linking recurring hotspots to planning and maintenance policies.

BIBLIOGRAPHY

1. Next.js Documentation. Vercel. Available at: <https://nextjs.org/docs>
2. React Documentation. Meta. Available at: <https://react.dev>
3. Express.js Documentation. Available at: <https://expressjs.com>
4. FastAPI Documentation. Available at: <https://fastapi.tiangolo.com>
5. Prisma ORM Documentation. Available at: <https://www.prisma.io/docs>
6. Expo Documentation. Available at: <https://docs.expo.dev>
7. LangChain Documentation. Available at: <https://python.langchain.com>
8. PostgreSQL Documentation. Available at: <https://www.postgresql.org/docs>
9. Bcrypt Documentation and community references for password hashing.
10. General software engineering references used for requirement analysis, system design, and workflow modeling.

APPENDIX

A. Important API Endpoints

Method	Endpoint	Purpose
POST	/api/auth/register	Register a new user
POST	/api/auth/login	Login an existing user
POST	/api/complaints	Create a complaint and attach AI analysis
GET	/api/complaints	List complaints
GET	/api/complaints/:id	Get complaint details
GET	/api/admin/overview	Administrative statistics summary
GET	/api/admin/complaints	Administrative complaint list
GET	/api/admin/users	Administrative user list
GET	/api/map/hotspots	Complaint hotspot information
POST	/analyze	AI microservice complaint analysis

B. Note on Final Submission Editing

Before formal university submission, the student should replace placeholder items on the cover and certificate pages with the final supervisor name, institution details, enrollment number, and any official formatting elements required by the department. The technical content, diagrams, and chapter structure in this report have been aligned to the dissertation format recovered from the provided PDF template.

Prepared from the UrbanEye project codebase and dissertation template structure.

APPENDIX: EXTENDED SUPPORTING DOCUMENTATION

A. Detailed Stakeholder Analysis

A complete final-year dissertation benefits from showing not only what the software does, but also who it serves and why each module exists. UrbanEye touches multiple stakeholders even in its prototype form. The resident cares about easy reporting and trustworthy tracking. The administrator cares about volume, trends, and issue clustering. The maintainer cares about service boundaries and resilience. The future field worker cares about actionability. This layered stakeholder view explains why the project evolved into a multi-surface platform rather than remaining a single complaint form.

Item	Description	Relevance to UrbanEye
Citizen / Resident	Primary end user who reports civic issues and tracks progress.	Shapes complaint intake, dashboard clarity, and trust.
Authority / Admin	Department-side reviewer who monitors patterns and complaint distribution.	Requires analytics, priority visibility, and hotspot summaries.
Field Staff	Potential future user responsible for actual on-ground issue resolution.	Informs future workflow expansion and assignment logic.
System Maintainer	Developer or operator maintaining APIs, AI service, and deployments.	Needs modular code, fallback support, and clear architecture.
Public Communication Team	Would use generated X-ready updates for official outreach.	Motivates social post generation and communication consistency.

B. Expanded User Manual

B.1 Web Application Usage Flow

1. Open the landing page and review the summary cards that describe overall complaint activity.
2. Navigate to the dashboard to access complaint cards, the report panel, and the admin-style overview metrics.
3. Use the report panel to submit an issue by entering title, description, and location.
4. Review complaint cards to observe department mapping, priority labels, and suggested actions.
5. Open map and profile-related screens for broader oversight and account-level browsing.

B.2 Mobile Application Usage Flow

1. Register or log in to access the resident workspace.
2. Use the home screen quick actions to jump to complaint reporting, dashboard, hotspot map, or profile.
3. File a complaint through the report screen and optionally attach an image.
4. Open the dashboard to see whether complaints are pending, in progress, or solved.
5. Save an X handle in the profile screen so the system can prepare communication aligned with the user preference.
6. Open complaint detail cards to see department, priority, location, and AI-generated social update text.

C. Requirements Traceability Matrix

Requirement	Description	Mapped Implementation
FR-1 Authentication	Register and login interfaces	backend/src/routes/authRoutes.js, backend/src/controllers/authcontroller.js, mobile_app/src/services/authService.js
FR-2 Complaint Submission	Web/mobile complaint capture	web_app/src/components/ReportPanel.tsx, mobile_app/src/screens/ReportScreen.js
FR-3 AI Complaint Analysis	Structured complaint enrichment	ai_service/app/main.py, ai_service/agents/social_agent.py, backend/src/services/aiService.js
FR-4 Complaint Tracking	Status-based follow-up	mobile_app/src/screens/DashbboardScreen.js, mobile_app/src/screens/ComplaintDetailScreen.js
FR-5 Admin Overview	Complaint statistics and summaries	backend/src/controllers/adminController.js, web_app/src/components/DashboardShell.tsx
FR-6 Hotspot Visibility	Location-oriented issue visibility	backend/src/routes/mapRoutes.js, web_app/src/components/MapComplaint.tsx
FR-7 Persistence	Store complaint metadata	backend/prisma/schema.prisma, backend/src/controllers/complaintController.js
FR-8 Fallback Resilience	Offline/demo continuity	backend/src/services/aiService.js, mobile_app/src/services/complaintService.js,

		web_app/src/services/complaintService.ts
--	--	--

D. Expanded Testing Matrix

Module	Test Focus	Rationale
Authentication	Valid registration, duplicate email, wrong password, fallback login	Ensures user access flow remains dependable.
Complaint API	Valid complaint, missing fields, fetch all, fetch by id	Verifies primary business flow.
AI Analysis	Sanitation, road, water, electrical, urgent keywords	Checks routing and priority inference.
Web Dashboard	Complaint rendering, report submission, overview load	Checks operator-side monitoring experience.
Mobile Dashboard	Status filter, solved rate, local complaint merge	Checks resident-side status visibility.
Persistence	Prisma schema generation, mock fallback, AsyncStorage recovery	Confirms resilience under mixed environments.
UI Validation	Required-field alerts, empty-state cards, navigation flows	Improves usability and prevents broken interactions.

E. Deployment and Execution Notes

The project can be executed as loosely coupled services. The backend API runs separately from the FastAPI AI service, while the web and mobile clients consume the backend over HTTP. This separation simplifies both testing and deployment. The backend is designed to run with mock data or a PostgreSQL-ready schema. The mobile app uses Expo for development efficiency, and the web app uses Next.js for rapid interface iteration.

A practical deployment path for UrbanEye would involve hosting the web frontend on a service optimized for React applications, running the Node backend behind a secure API gateway, exposing the FastAPI service as an internal or protected microservice, and using PostgreSQL for durable storage. The current system already separates concerns in a way that makes this migration straightforward.

F. API Documentation Reference

Method	Path	Purpose	Typical Input
POST	/api/auth/register	Register a resident account	Name, email, password
POST	/api/auth/login	Authenticate existing user	Email, password
POST	/api/complaints	Create a complaint and enrich it through AI	Title, description, location, optional image
GET	/api/complaints	List complaints	No body required
GET	/api/complaints/:id	Fetch complaint detail	Complaint ID
GET	/api/admin/overview	Fetch analytics summary	No body required
GET	/api/admin/complaints	Fetch admin complaint list	No body required
GET	/api/admin/users	Fetch admin user list	No body required
GET	/api/map/hotspots	Fetch hotspot map records	No body required
POST	/analyze	Analyze complaint text using AI service	Title, description, location

G. Expanded Design Discussion

One of the strengths of UrbanEye is that its AI layer does not replace application logic; instead, it complements it. The backend retains ownership of validation, persistence, and shaping of complaint records, while the AI layer enriches those records. This division improves trustworthiness because the deterministic parts of the system remain explicit and testable. It also improves maintainability because AI logic can evolve without forcing a full rewrite of the complaint API.

The frontend architecture similarly reflects task-specific design. The web application is oriented toward overview and operational visibility, whereas the mobile application is oriented toward immediacy and follow-up. The mobile dashboard's solved-versus-open framing is especially important in civic use cases because users care less about abstract analytics and more about whether the issue has been addressed. This is why state wording, filter chips, and detail cards become significant design decisions rather than mere styling choices.

From a software engineering perspective, fallback support is another important design decision. In many student projects, systems fail entirely when an external API or database is missing. UrbanEye deliberately avoids this by supporting mock stores, local persistence, and local analysis. This not only helps during demonstration, but also reflects good fault-tolerant design practice.

H. Source Code Listings

The following code listings are included as supporting evidence of implementation. They document the actual modules used in the project and are helpful for viva discussion, technical review, and future maintenance. Including these listings also makes the report more complete as a final-year dissertation artifact because the reader can directly connect design claims with implementation details.

Backend Application Bootstrap

File: backend/src/app.js

```
import express from "express";
import cors from "cors";
import cookieParser from "cookie-parser";

import authRoutes from "../routes/authRoutes.js";
import complaintRoutes from "../routes/complaintRoutes.js";
import adminRoutes from "../routes/adminRoutes.js";
import mapRoutes from "../routes/mapRoutes.js";
import userRoutes from "../routes/userRoutes.js";

const app = express();

app.use(
  cors({
    origin: true,
    credentials: true,
  }),
);
app.use(express.json());
app.use(cookieParser());

app.get("/", (req, res) => {
  res.json({
    ok: true,
    name: "UrbanEye API",
    message: "API running",
  });
});

app.use("/api/auth", authRoutes);
app.use("/api/complaints", complaintRoutes);
app.use("/api/admin", adminRoutes);
app.use("/api/map", mapRoutes);
app.use("/api/users", userRoutes);

app.use((err, req, res, next) => {
  console.error(err);
  res.status(err.statusCode || 500).json({
    message: err.message || "Something went wrong",
```

```
    });  
  });  
  
  export default app;
```

Backend Server Entry

File: backend/src/server.js

```
import app from "../app.js";  
  
const PORT = Number(process.env.PORT) || 5000;  
  
app.listen(PORT, () => {  
  console.log(`Server running on port ${PORT}`);  
});
```

Authentication Routes

File: backend/src/routes/authRoutes.js

```
import express from "express";  
import { register, login } from "../controllers/authController.js";  
  
const router = express.Router();  
  
router.post("/register", register);  
router.post("/login", login);  
  
export default router;
```

Complaint Routes

File: backend/src/routes/complaintRoutes.js

```
import express from "express";  
import {  
  createComplaint,  
  getComplaint,  
  getComplaints,  
} from "../controllers/complaintController.js";  
  
const router = express.Router();
```

```
router.post("/", createComplaint);
router.get("/", getComplaints);
router.get("/:id", getComplaint);

export default router;
```

Admin Routes

File: backend/src/routes/adminRoutes.js

```
import { Router } from "express";
import {
  getAdminComplaints,
  getAdminOverview,
  getAdminUsers,
} from "../controllers/adminController.js";

const router = Router();

router.get("/overview", getAdminOverview);
router.get("/complaints", getAdminComplaints);
router.get("/users", getAdminUsers);

export default router;
```

Map Routes

File: backend/src/routes/mapRoutes.js

```
import { Router } from "express";
import { getMapHotspotsController } from "../controllers/mapController.js";

const router = Router();

router.get("/hotspots", getMapHotspotsController);

export default router;
```

Authentication Controller

File: backend/src/controllers/authcontroller.js

```
import bcrypt from "bcrypt";

import prisma from "../utils/prisma.js";
import { addUser, findUserByEmail } from "../data/mockStore.js";
```

```

function sanitizeUser(user) {
  if (!user) {
    return null;
  }

  const { password, ...safeUser } = user;
  return safeUser;
}

function buildUserPayload({ name, email, passwordHash, role = "CITIZEN" }) {
  return {
    id: crypto.randomUUID(),
    name,
    email,
    password: passwordHash,
    role,
    createdAt: new Date().toISOString(),
  };
}

export const register = async (req, res, next) => {
  try {
    const { name, email, password } = req.body;

    if (!name || !email || !password) {
      return res.status(400).json({ message: "Name, email, and password are required." });
    }

    const normalizedEmail = email.trim().toLowerCase();
    const passwordHash = await bcrypt.hash(password, 10);

    if (prisma && process.env.DATABASE_URL) {
      const existingUser = await prisma.user.findUnique({
        where: { email: normalizedEmail },
      });

      if (existingUser) {
        return res.status(409).json({ message: "User already exists." });
      }

      const user = await prisma.user.create({
        data: {
          name: name.trim(),
          email: normalizedEmail,
          password: passwordHash,
        },
      });

      return res.status(201).json({ user: sanitizeUser(user) });
    }
  }
}

```

```

const existingUser = findUserByEmail(normalizedEmail);
if (existingUser) {
  return res.status(409).json({ message: "User already exists." });
}

const user = addUser(
  buildUserPayload({
    name: name.trim(),
    email: normalizedEmail,
    passwordHash,
  }),
);

return res.status(201).json({ user: sanitizeUser(user) });
} catch (error) {
  return next(error);
}
};

export const login = async (req, res, next) => {
  try {
    const { email, password } = req.body;

    if (!email || !password) {
      return res.status(400).json({ message: "Email and password are required."
});
    }

    const normalizedEmail = email.trim().toLowerCase();

    if (prisma && process.env.DATABASE_URL) {
      const user = await prisma.user.findUnique({
        where: { email: normalizedEmail },
      });

      if (!user) {
        return res.status(401).json({ message: "Invalid credentials." });
      }

      const isMatch = await bcrypt.compare(password, user.password);
      if (!isMatch) {
        return res.status(401).json({ message: "Invalid credentials." });
      }

      return res.json({ user: sanitizeUser(user) });
    }

    const user = findUserByEmail(normalizedEmail);
    if (!user) {
      return res.status(401).json({ message: "Invalid credentials." });
    }
  }
};

```

```

    const isMatch = await bcrypt.compare(password, user.password);
    if (!isMatch) {
      return res.status(401).json({ message: "Invalid credentials." });
    }

    return res.json({ user: sanitizeUser(user) });
  } catch (error) {
    return next(error);
  }
};

```

Complaint Controller

File: backend/src/controllers/complaintController.js

```

import prisma from "../utils/prisma.js";
import {
  addComplaint,
  getComplaintById,
  getRawUsers,
  listComplaints,
} from "../data/mockStore.js";
import { analyzeComplaint } from "../services/aiService.js";

function normalizeComplaintShape(complaint) {
  return {
    ...complaint,
    category: complaint.category || "General",
    priority: complaint.priority || "MEDIUM",
    department: complaint.department || "Civic Response Cell",
    sentiment: complaint.sentiment || "concerned",
    confidence: complaint.confidence ?? 0.72,
    suggestedAction: complaint.suggestedAction || complaint.suggested_action ||
    "",
    socialPost: complaint.socialPost || complaint.social_post || "",
    image: complaint.image || "",
    latitude: Number(complaint.latitude) || 25.5941,
    longitude: Number(complaint.longitude) || 85.1376,
    upvotes: Number(complaint.upvotes) || 0,
  };
};

export const createComplaint = async (req, res, next) => {
  try {
    const { title, description, location, image, latitude, longitude } = req.body;

    if (!title || !description || !location) {
      return res.status(400).json({ message: "Title, description, and location are required." });
    }
  }
};

```

```

}

const analysis = await analyzeComplaint({ title, description, location });

if (prisma && process.env.DATABASE_URL) {
  const firstUser = await prisma.user.findFirst();

  if (!firstUser) {
    return res.status(400).json({ message: "Create a user before filing a complaint." });
  }

  const complaint = await prisma.complaint.create({
    data: {
      title: title.trim(),
      description: description.trim(),
      location: location.trim(),
      category: analysis.category,
      priority: analysis.priority,
      department: analysis.department,
      sentiment: analysis.sentiment,
      confidence: analysis.confidence,
      suggestedAction: analysis.suggestedAction,
      socialPost: analysis.socialPost,
      image: image || "",
      latitude: Number(latitude) || 25.5941,
      longitude: Number(longitude) || 85.1376,
      upvotes: 1,
      userId: firstUser.id,
    },
    include: {
      user: true,
    },
  });

  return res.status(201).json(
    normalizeComplaintShape(complaint),
  );
}

const defaultUser = getRawUsers()[0];
const complaint = addComplaint({
  id: crypto.randomUUID(),
  title: title.trim(),
  description: description.trim(),
  location: location.trim(),
  ...analysis,
  status: "PENDING",
  image: image || "",
  latitude: Number(latitude) || 25.5941,
  longitude: Number(longitude) || 85.1376,
  upvotes: 1,

```

```

        userId: defaultUser.id,
        createdAt: new Date().toISOString(),
    });

    return res.status(201).json(normalizeComplaintShape(complaint));
} catch (error) {
    return next(error);
}
};

export const getComplaints = async (req, res, next) => {
    try {
        if (prisma && process.env.DATABASE_URL) {
            const complaints = await prisma.complaint.findMany({
                include: {
                    user: true,
                },
                orderBy: {
                    createdAt: "desc",
                },
            });

            return res.json(complaints.map(normalizeComplaintShape));
        }

        return res.json(listComplaints().map(normalizeComplaintShape));
    } catch (error) {
        return next(error);
    }
};

export const getComplaint = async (req, res, next) => {
    try {
        if (prisma && process.env.DATABASE_URL) {
            const complaint = await prisma.complaint.findUnique({
                where: {
                    id: req.params.id,
                },
                include: {
                    user: true,
                },
            });

            if (!complaint) {
                return res.status(404).json({ message: "Complaint not found." });
            }

            return res.json(normalizeComplaintShape(complaint));
        }

        const complaint = getComplaintById(req.params.id);
    }
};

```

```
    if (!complaint) {
      return res.status(404).json({ message: "Complaint not found." });
    }

    return res.json(normalizeComplaintShape(complaint));
  } catch (error) {
    return next(error);
  }
};
```

Admin Controller

File: backend/src/controllers/adminController.js

```
import { getAnalytics, listComplaints, listUsers } from "../data/mockStore.js";

export function getAdminOverview(req, res) {
  res.json(getAnalytics());
}

export function getAdminComplaints(req, res) {
  res.json(listComplaints());
}

export function getAdminUsers(req, res) {
  res.json(listUsers());
}
```

Map Controller

File: backend/src/controllers/mapController.js

```
import { getMapHotspots } from "../data/mockStore.js";

export function getMapHotspotsController(req, res) {
  res.json(getMapHotspots());
}
```

Backend AI Service Adapter

File: backend/src/services/aiService.js

```
const AI_SERVICE_URL = process.env.AI_SERVICE_URL?.replace(/\/$/ , "") ||
"http://localhost:8000";

const SOCIAL_HASHTAGS = {
```

```

    Sanitation: ["#CleanStreets", "#CivicAction"],
    Electricity: ["#StreetlightFix", "#CivicAction"],
    Water: ["#WaterAlert", "#CivicAction"],
    Roads: ["#RoadSafety", "#CivicAction"],
    General: ["#CityUpdate", "#CivicAction"],
};

function keywordMatch(text, keywords) {
    return keywords.some((keyword) => text.includes(keyword));
}

function compactText(text = "") {
    return text.replace(/\s+/g, " ").trim();
}

function trimToLength(text, limit) {
    const compact = compactText(text);
    if (compact.length <= limit) {
        return compact;
    }

    const trimmed = compact.slice(0, Math.max(limit - 1, 0)).replace(/[,.;:-]+$/, "");
    return `${trimmed}...`;
}

function buildSocialPost({ title = "", description = "", location = "", category = "General", priority = "MEDIUM", department = "Civic Response Cell" }) {
    const prefixByPriority = {
        CRITICAL: "Urgent civic alert:",
        HIGH: "High-priority civic alert:",
        MEDIUM: "Civic update:",
        LOW: "Civic update:",
    };
    const hashtags = (SOCIAL_HASHTAGS[category] || SOCIAL_HASHTAGS.General).join(" ");
    const issueSummary = trimToLength(description || title, 85);

    return trimToLength(
        `${prefixByPriority[priority] || prefixByPriority.MEDIUM} ${title} at ${location}. ${issueSummary} Assigned to ${department}. ${hashtags}`,
        280,
    );
}

function localAnalysis({ title = "", description = "", location = "" }) {
    const normalized = `${title} ${description} ${location}`.toLowerCase();

    let category = "General";
    let department = "Civic Response Cell";
    let priority = "MEDIUM";

```

```

    if (keywordMatch(normalized, ["garbage", "waste", "trash"])) {
      category = "Sanitation";
      department = "Sanitation Department";
      priority = "HIGH";
    } else if (keywordMatch(normalized, ["light", "electric", "power"])) {
      category = "Electricity";
      department = "Electricity Department";
    } else if (keywordMatch(normalized, ["water", "leak", "pipeline"])) {
      category = "Water";
      department = "Water Department";
      priority = "HIGH";
    } else if (keywordMatch(normalized, ["pothole", "road", "traffic", "drainage"])) {
      category = "Roads";
      department = "Road Department";
      priority = "HIGH";
    }

    if (keywordMatch(normalized, ["fire", "accident", "flood", "unsafe"])) {
      priority = "CRITICAL";
    }

    return {
      category,
      priority,
      department,
      sentiment: priority === "CRITICAL" ? "urgent" : "concerned",
      confidence: 0.72,
      suggestedAction: `Route this complaint to ${department} for field
validation.`,
      socialPost: buildSocialPost({
        title,
        description,
        location,
        category,
        priority,
        department,
      }),
    };
  }
}

export async function analyzeComplaint(payload) {
  try {
    const response = await fetch(`${AI_SERVICE_URL}/analyze`, {
      method: "POST",
      headers: {
        "Content-Type": "application/json",
      },
      body: JSON.stringify(payload),
    });

    if (!response.ok) {

```

```
        throw new Error("AI service unavailable");
    }

    return await response.json();
} catch (error) {
    return localAnalysis(payload);
}
}
```

Mock Store Data Layer

File: backend/src/data/mockStore.js

```
const users = [
  {
    id: "seed-user-1",
    name: "Aarav Singh",
    email: "citizen@urbaneye.dev",
    password: "$2b$10$9cuvx0ExNkQWXdQ2n6A0n.6M3NSpQqVQ8jQJ4x6cPX0T3CB7wWQ1K",
    role: "CITIZEN",
    createdAt: "2026-03-18T08:20:00.000Z",
  },
  {
    id: "seed-user-2",
    name: "Meera Rao",
    email: "authority@urbaneye.dev",
    password: "$2b$10$9cuvx0ExNkQWXdQ2n6A0n.6M3NSpQqVQ8jQJ4x6cPX0T3CB7wWQ1K",
    role: "AUTHORITY",
    createdAt: "2026-03-18T09:00:00.000Z",
  },
  {
    id: "seed-user-3",
    name: "Sonal Verma",
    email: "admin@urbaneye.dev",
    password: "$2b$10$9cuvx0ExNkQWXdQ2n6A0n.6M3NSpQqVQ8jQJ4x6cPX0T3CB7wWQ1K",
    role: "ADMIN",
    createdAt: "2026-03-18T09:30:00.000Z",
  },
];

const complaints = [
  {
    id: "cmp-1001",
    title: "Overflowing roadside garbage",
    description: "Garbage bins near Sector 8 market have not been cleared for three days.",
    location: "Sector 8 Market, Patna",
    category: "Sanitation",
    priority: "HIGH",
    status: "IN_PROGRESS",
  },
];
```

```
department: "Sanitation Department",
sentiment: "frustrated",
confidence: 0.89,
suggestedAction: "Dispatch sanitation pickup team within 12 hours.",
socialPost: "Ticket escalated for sanitation crew update.",
image: "",
latitude: 25.6175,
longitude: 85.1452,
upvotes: 19,
userId: "seed-user-1",
createdAt: "2026-03-22T09:30:00.000Z",
},
{
  id: "cmp-1002",
  title: "Streetlight outage",
  description: "Two consecutive streetlights are out near the school crossing.",
  location: "Boring Road Crossing, Patna",
  category: "Electricity",
  priority: "MEDIUM",
  status: "PENDING",
  department: "Electricity Department",
  sentiment: "concerned",
  confidence: 0.78,
  suggestedAction: "Assign to night maintenance team for inspection.",
  socialPost: "",
  image: "",
  latitude: 25.6128,
  longitude: 85.1178,
  upvotes: 11,
  userId: "seed-user-1",
  createdAt: "2026-03-21T18:10:00.000Z",
},
{
  id: "cmp-1003",
  title: "Water leakage near bus stop",
  description: "Continuous water leakage is flooding the footpath near the bus
stop.",
  location: "Kankarbagh Main Road, Patna",
  category: "Water",
  priority: "HIGH",
  status: "PENDING",
  department: "Water Department",
  sentiment: "urgent",
  confidence: 0.84,
  suggestedAction: "Inspect pipeline pressure and send repair crew.",
  socialPost: "",
  image: "",
  latitude: 25.5944,
  longitude: 85.1612,
  upvotes: 23,
  userId: "seed-user-1",
  createdAt: "2026-03-23T06:45:00.000Z",
```

```

    },
    {
      id: "cmp-1004",
      title: "Pothole causing traffic slowdown",
      description: "A deep pothole at the flyover approach is forcing bikes into
traffic.",
      location: "Bailey Road Flyover, Patna",
      category: "Roads",
      priority: "HIGH",
      status: "RESOLVED",
      department: "Road Department",
      sentiment: "annoyed",
      confidence: 0.92,
      suggestedAction: "Mark area and schedule resurfacing crew.",
      socialPost: "Resolved after emergency patch work.",
      image: "",
      latitude: 25.6096,
      longitude: 85.1081,
      upvotes: 31,
      userId: "seed-user-1",
      createdAt: "2026-03-19T07:50:00.000Z",
    },
  ],

function decorateComplaint(complaint) {
  return {
    ...complaint,
    user: users.find((user) => user.id === complaint.userId) || null,
  };
}

export function listUsers() {
  return users.map(({ password, ...user }) => user);
}

export function getRawUsers() {
  return users;
}

export function findUserByEmail(email) {
  return users.find((user) => user.email.toLowerCase() === email.toLowerCase()) ||
null;
}

export function addUser(user) {
  users.push(user);
  return user;
}

export function listComplaints() {
  return complaints
    .map(decorateComplaint)

```

```

        .sort((a, b) => new Date(b.createdAt) - new Date(a.createdAt));
    }

export function getComplaintById(id) {
    const complaint = complaints.find((item) => item.id === id);
    return complaint ? decorateComplaint(complaint) : null;
}

export function addComplaint(complaint) {
    complaints.unshift(complaint);
    return decorateComplaint(complaint);
}

export function getAnalytics() {
    const items = listComplaints();
    const statusBreakdown = items.reduce((acc, item) => {
        acc[item.status] = (acc[item.status] || 0) + 1;
        return acc;
    }, {});
    const priorityBreakdown = items.reduce((acc, item) => {
        acc[item.priority] = (acc[item.priority] || 0) + 1;
        return acc;
    }, {});
    const departmentBreakdown = items.reduce((acc, item) => {
        acc[item.department] = (acc[item.department] || 0) + 1;
        return acc;
    }, {});

    return {
        totals: {
            complaints: items.length,
            pending: statusBreakdown.PENDING || 0,
            inProgress: statusBreakdown.IN_PROGRESS || 0,
            resolved: statusBreakdown.RESOLVED || 0,
            users: users.length,
        },
        statusBreakdown,
        priorityBreakdown,
        departmentBreakdown,
        recentComplaints: items.slice(0, 5),
    };
}

export function getMapHotspots() {
    return listComplaints().map((item) => ({
        id: item.id,
        title: item.title,
        location: item.location,
        latitude: item.latitude,
        longitude: item.longitude,
        category: item.category,
        priority: item.priority,
    }));
}

```

```
        status: item.status,
    }));
}
```

Prisma Utility

File: backend/src/utils/prisma.js

```
let prisma = null;

if (process.env.DATABASE_URL) {
  try {
    const prismaPackage = await import("@prisma/client");
    const PrismaClient = prismaPackage.PrismaClient ||
      prismaPackage.default?.PrismaClient;

    prisma = PrismaClient ? new PrismaClient() : null;
  } catch (error) {
    prisma = null;
  }
}

export default prisma;
```

Prisma Schema

File: backend/prisma/schema.prisma

```
generator client {
  provider = "prisma-client-js"
}

datasource db {
  provider = "postgresql"
}

model User {
  id          String    @id @default(uuid())
  name        String
  email       String    @unique
  password    String
  role        Role      @default(CITIZEN)
  createdAt   DateTime  @default(now())

  complaints Complaint[]
}

model Complaint {
```

```

    id            String    @id @default(uuid())
    title         String
    description    String
    location       String
    category       String    @default("General")
    priority       String    @default("MEDIUM")
    status         Status    @default(PENDING)
    department     String    @default("Civic Response Cell")
    sentiment      String    @default("concerned")
    confidence     Float     @default(0.0)
    suggestedAction String @default("")
    socialPost     String    @default("")
    image          String    @default("")
    latitude       Float     @default(25.5941)
    longitude      Float     @default(85.1376)
    upvotes        Int       @default(0)

    userId         String
    user           User      @relation(fields: [userId], references: [id])

    createdAt      DateTime @default(now())
}

enum Role {
    CITIZEN
    ADMIN
    AUTHORITY
}

enum Status {
    PENDING
    IN_PROGRESS
    RESOLVED
}

```

AI Service Main Application

File: ai_service/app/main.py

```

from fastapi import FastAPI
from pydantic import BaseModel, Field

from agents.social_agent import generate_social_post

class ComplaintPayload(BaseModel):
    title: str = Field(..., min_length=3)
    description: str = Field(..., min_length=5)
    location: str = Field(..., min_length=3)

```

```

class AnalysisResult(BaseModel):
    category: str
    priority: str
    department: str
    sentiment: str
    confidence: float
    suggestedAction: str
    socialPost: str

def keyword_match(text: str, keywords: list[str]) -> bool:
    return any(keyword in text for keyword in keywords)

def analyze_locally(payload: ComplaintPayload) -> AnalysisResult:
    text = f"{payload.title} {payload.description} {payload.location}".lower()

    category = "General"
    department = "Civic Response Cell"
    priority = "MEDIUM"
    sentiment = "concerned"

    if keyword_match(text, ["garbage", "waste", "trash"]):
        category = "Sanitation"
        department = "Sanitation Department"
        priority = "HIGH"
    elif keyword_match(text, ["light", "power", "electric"]):
        category = "Electricity"
        department = "Electricity Department"
    elif keyword_match(text, ["water", "leak", "pipeline"]):
        category = "Water"
        department = "Water Department"
        priority = "HIGH"
    elif keyword_match(text, ["pothole", "road", "traffic", "drainage"]):
        category = "Roads"
        department = "Road Department"
        priority = "HIGH"

    if keyword_match(text, ["fire", "accident", "flood", "unsafe"]):
        priority = "CRITICAL"
        sentiment = "urgent"

    action = f"Route this complaint to {department} and request field validation."
    social = generate_social_post(
        title=payload.title,
        description=payload.description,
        location=payload.location,
        category=category,
        priority=priority,
        department=department,
    )

```

```

        return AnalysisResult(
            category=category,
            priority=priority,
            department=department,
            sentiment=sentiment,
            confidence=0.74,
            suggestedAction=action,
            socialPost=social,
        )

app = FastAPI(
    title="UrbanEye AI Service",
    description="Complaint analysis and triage service for civic issue routing.",
    version="1.0.0",
)

@app.get("/health")
def health_check():
    return {"ok": True, "service": "urbaneye-ai"}

@app.post("/analyze", response_model=AnalysisResult)
def analyze_complaint(payload: ComplaintPayload):
    return analyze_locally(payload)

```

AI Social Agent

File: ai_service/agents/social_agent.py

```

from __future__ import annotations

import os
import re
from typing import Any

MAX_X_POST_LENGTH = 280

CATEGORY_HASHTAGS = {
    "Sanitation": ["#CleanStreets", "#CivicAction"],
    "Electricity": ["#StreetlightFix", "#CivicAction"],
    "Water": ["#WaterAlert", "#CivicAction"],
    "Roads": ["#RoadSafety", "#CivicAction"],
    "General": ["#CityUpdate", "#CivicAction"],
}

def _compact_text(value: str) -> str:

```

```

return re.sub(r"\s+", " ", value).strip()

def _trim_to_length(value: str, limit: int) -> str:
    compact = _compact_text(value)
    if len(compact) <= limit:
        return compact
    trimmed = compact[: max(limit - 1, 0)].rstrip(" ,.:-")
    return f"{trimmed}..." if trimmed else compact[:limit]

def _hashtags_for_category(category: str) -> str:
    tags = CATEGORY_HASHTAGS.get(category, CATEGORY_HASHTAGS["General"])
    return " ".join(tags)

def build_fallback_social_post(
    *,
    title: str,
    description: str,
    location: str,
    category: str,
    priority: str,
    department: str,
) -> str:
    priority_prefix = {
        "CRITICAL": "Urgent civic alert:",
        "HIGH": "High-priority civic alert:",
        "MEDIUM": "Civic update:",
        "LOW": "Civic update:",
    }.get(priority, "Civic update:")

    issue_summary = _trim_to_length(description or title, 85)
    hashtags = _hashtags_for_category(category)

    post = (
        f"{priority_prefix} {title} at {location}. "
        f"{issue_summary} Assigned to {department}. {hashtags}"
    )
    return _trim_to_length(post, MAX_X_POST_LENGTH)

def _llm_is_configured() -> bool:
    return bool(os.getenv("LLM_MODEL") and os.getenv("LLM_PROVIDER"))

def _extract_response_text(response: Any) -> str:
    if hasattr(response, "content"):
        return _compact_text(str(response.content))
    return _compact_text(str(response))

```

```

def generate_social_post(
    *,
    title: str,
    description: str,
    location: str,
    category: str,
    priority: str,
    department: str,
) -> str:
    fallback = build_fallback_social_post(
        title=title,
        description=description,
        location=location,
        category=category,
        priority=priority,
        department=department,
    )

    if not _llm_is_configured():
        return fallback

    try:
        from langchain_core.prompts import ChatPromptTemplate

        from config.llm import get_llm

        llm = get_llm()
        prompt = ChatPromptTemplate.from_messages(
            [
                (
                    "system",
                    (
                        "You write concise X posts for civic issue automation. "
                        f"Return exactly one post under {MAX_X_POST_LENGTH}
characters. "
                        "Mention the issue and location, keep the tone factual and
public-safe, "
                        "and end with two short relevant hashtags. No markdown
fences."
                    ),
                ),
                (
                    "human",
                    (
                        "Title: {title}\n"
                        "Description: {description}\n"
                        "Location: {location}\n"
                        "Category: {category}\n"
                        "Priority: {priority}\n"
                        "Assigned department: {department}"
                    ),
                ),
            ],
        ),

```

```

        ]
    )
    chain = prompt | llm
    response = chain.invoke(
        {
            "title": title,
            "description": description,
            "location": location,
            "category": category,
            "priority": priority,
            "department": department,
        }
    )
    candidate = _trim_to_length(_extract_response_text(response),
MAX_X_POST_LENGTH)
    return candidate or fallback
except Exception:
    return fallback

```

AI LLM Configuration

File: ai_service/config/llm.py

```

from langchain.chat_models import init_chat_model
from dotenv import load_dotenv
import os
load_dotenv()

def get_llm():
    model=os.getenv('LLM_MODEL')
    provider=os.getenv('LLM_PROVIDER')

    llm= init_chat_model(
        model=model,
        model_provider=provider,
    )
    return llm

```

Web Home Page

File: web_app/src/app/page.tsx

```

import Link from "next/link";

import Navbar from "../components/Navbar";
import ComplaintCard from "../components/Complaintcard";
import Sidebar from "../components/Sidebar";
import { demoComplaints } from "../lib/demoData";

```

```

const stats = [
  { label: "Open complaints", value: "128" },
  { label: "Resolved this week", value: "43" },
  { label: "Avg response time", value: "7.2h" },
];

export default function Home() {
  return (
    <div className="pb-14">
      <Navbar />
      <main className="mx-auto flex max-w-6xl flex-col gap-8 px-6 md:px-10">
        <section className="section-grid items-start">
          <div className="glass-card rounded-[40px] p-8 md:p-10">
            <p className="text-sm font-semibold uppercase tracking-[0.28em] text-[var(--accent-dark)]">
              Civic issue resolution system
            </p>
            <h1 className="mt-4 max-w-3xl text-5xl font-semibold leading-tight">
              Report city problems, route them faster, and keep residents
informed.
            </h1>
            <p className="mt-5 max-w-2xl text-lg leading-8 text-[var(--ink-muted)]">
              UrbanEye brings together citizen reporting, admin visibility, and
AI-assisted triage
              in the same workflow without changing your current stack.
            </p>
            <div className="mt-8 flex flex-wrap gap-4">
              <Link
                href="/dashboard"
                className="rounded-full bg-[var(--foreground)] px-6 py-3 text-sm
font-semibold text-white transition hover:bg-[var(--accent-dark)]"
              >
                Open Dashboard
              </Link>
              <Link
                href="/register"
                className="rounded-full border border-[var(--border)] bg-white/75
px-6 py-3 text-sm font-semibold"
              >
                Create Citizen Account
              </Link>
            </div>
            <div className="mt-10 grid gap-4 md:grid-cols-3">
              {stats.map((stat) => (
                <div key={stat.label} className="rounded-[28px] border border-[var(--border)] bg-white/65 p-5">
                  <p className="text-sm text-[var(--ink-muted)]">{stat.label}</p>
                  <p className="mt-3 text-3xl font-semibold">{stat.value}</p>
                </div>
              ))}
            </div>
          </div>
        </section>
      </main>
    </div>
  );
}

```

```

        </div>
      </div>
      <Sidebar />
    </section>

    <section className="grid gap-5 md:grid-cols-3">
      {demoComplaints.map((complaint) => (
        <ComplaintCard key={complaint.id} complaint={complaint} />
      ))}
    </section>
  </main>
</div>
);
}

```

Web Dashboard Page

File: web_app/src/app/dashboard/page.tsx

```

import DashboardShell from "../../components/DashboardShell";
import Navbar from "../../components/Navbar";
import Sidebar from "../../components/Sidebar";

export default function DashboardPage() {
  return (
    <div className="pb-12">
      <Navbar />
      <main className="mx-auto flex max-w-6xl flex-col gap-8 px-6 md:px-10">
        <section className="section-grid">
          <div className="glass-card rounded-[36px] p-8">
            <p className="text-xs font-semibold uppercase tracking-[0.24em] text-[var(--ink-muted)]">
              Operations board
            </p>
            <h1 className="mt-3 text-4xl font-semibold">Today's complaint
              flow at a glance</h1>
            <p className="mt-4 max-w-2xl text-[var(--ink-muted)]">
              Track incoming complaints, identify hotspots, and keep response
              teams aligned from
              one place.
            </p>
          </div>
          <Sidebar />
        </section>
      <DashboardShell />
    </main>
    </div>
  );
}

```

Web Profile Page

File: web_app/src/app/profile/page.tsx

```
import Navbar from "../../components/Navbar";

export default function ProfilePage() {
  return (
    <div className="pb-12">
      <Navbar />
      <main className="mx-auto max-w-4xl px-6 md:px-10">
        <section className="glass-card rounded-[36px] p-8">
          <p className="text-xs font-semibold uppercase tracking-[0.24em] text-[var(--ink-muted)]">
            Citizen profile
          </p>
          <h1 className="mt-3 text-4xl font-semibold">Resident account
            snapshot</h1>
          <div className="mt-8 grid gap-4 md:grid-cols-2">
            <div className="rounded-[28px] border border-[var(--border)] bg-white/70 p-5">
              <p className="text-sm text-[var(--ink-muted)]">Name</p>
              <p className="mt-2 text-xl font-semibold">Aarav Singh</p>
            </div>
            <div className="rounded-[28px] border border-[var(--border)] bg-white/70 p-5">
              <p className="text-sm text-[var(--ink-muted)]">Email</p>
              <p className="mt-2 text-xl font-semibold">citizen@urbaneye.dev</p>
            </div>
          </div>
        </section>
      </main>
    </div>
  );
}
```

Web Dashboard Shell

File: web_app/src/components/DashboardShell.tsx

```
"use client";

import { useEffect, useState } from "react";

import ComplaintCard from "../Complaintcard";
import MapComplaint from "../MapComplaint";
import ReportPanel from "../ReportPanel";
import { type AdminOverview, type Complaint, demoOverview } from
  "../lib/demoData";
```

```

import { getAdminOverview } from "../services/adminService";
import { getComplaints } from "../services/complaintService";

export default function DashboardShell() {
  const [overview, setOverview] = useState<AdminOverview>(demoOverview);
  const [complaints, setComplaints] = useState<Complaint[]>(demoOverview.recentComplaints);

  const load = () => {
    Promise.all([getAdminOverview(), getComplaints()]).then(([nextOverview, nextComplaints]) => {
      setOverview(nextOverview);
      setComplaints(nextComplaints);
    });
  };

  useEffect(() => {
    void load();
  }, []);

  const stats = [
    { label: "Open complaints", value: overview.totals.complaints },
    { label: "Pending", value: overview.totals.pending },
    { label: "In progress", value: overview.totals.inProgress },
    { label: "Resolved", value: overview.totals.resolved },
  ];

  return (
    <div className="grid gap-8">
      <section className="grid gap-4 md:grid-cols-4">
        {stats.map((stat) => (
          <div key={stat.label} className="glass-card rounded-[28px] p-5">
            <p className="text-sm text-[var(--ink-muted)]">{stat.label}</p>
            <p className="mt-3 text-3xl font-semibold">{stat.value}</p>
          </div>
        ))}
      </section>
      <section className="section-grid">
        <ReportPanel onCreated={load} />
        <MapComplaint complaints={complaints.slice(0, 3)} />
      </section>
      <section className="grid gap-5 md:grid-cols-2">
        {complaints.map((complaint) => (
          <ComplaintCard key={complaint.id} complaint={complaint} />
        ))}
      </section>
    </div>
  );
}

```

Web Complaint Card

File: web_app/src/components/Complaintcard.tsx

```
import type { Complaint } from "../lib/demoData";

type Props = {
  complaint: Complaint;
};

export default function ComplaintCard({ complaint }: Props) {
  return (
    <article className="glass-card rounded-[28px] p-5">
      <div className="mb-4 flex items-start justify-between gap-4">
        <div>
          <p className="text-xs font-semibold uppercase tracking-[0.24em] text-[var(--ink-muted)]">
            {complaint.category}
          </p>
          <h3 className="mt-2 text-xl font-semibold">{complaint.title}</h3>
        </div>
        <span className="rounded-full bg-[rgba(42,157,143,0.12)] px-3 py-1 text-xs font-semibold text-[var(--success)]">
          {complaint.status.replace("_", " ")}
        </span>
      </div>
      <p className="text-sm leading-6 text-[var(--ink-muted)]">
        {complaint.description}</p>
      <div className="mt-5 flex flex-wrap gap-3 text-sm">
        <span className="rounded-full bg-white/70 px-3 py-1">{complaint.location}</span>
        <span className="rounded-full bg-[rgba(239,131,84,0.14)] px-3 py-1 text-[var(--accent-dark)]">
          Priority {complaint.priority}
        </span>
        {complaint.department ? (
          <span className="rounded-full bg-[rgba(20,33,61,0.08)] px-3 py-1">
            {complaint.department}</span>
        ) : null}
      </div>
      {complaint.suggestedAction ? (
        <p className="mt-4 text-sm leading-6 text-[var(--ink-muted)]">
          Suggested action: {complaint.suggestedAction}
        </p>
      ) : null}
    </article>
  );
}
```

Web Report Panel

File: web_app/src/components/ReportPanel.tsx

```

"use client";

import { useState } from "react";

import { createComplaint } from "../services/complaintService";

type Props = {
  onCreate?: () => void;
};

export default function ReportPanel({ onCreate }: Props) {
  const [form, setForm] = useState({
    title: "",
    description: "",
    location: "",
  });
  const [submitting, setSubmitting] = useState(false);
  const [message, setMessage] = useState("");

  const updateField = (key: keyof typeof form, value: string) => {
    setForm((current) => ({ ...current, [key]: value }));
  };

  const handleSubmit = async (event: React.FormEvent<HTMLFormElement>) => {
    event.preventDefault();
    setSubmitting(true);
    setMessage("");

    try {
      await createComplaint(form);
      setMessage("Complaint submitted and routed for triage.");
      setForm({ title: "", description: "", location: "" });
      onCreate?.();
    } catch {
      setMessage("Unable to submit complaint right now.");
    } finally {
      setSubmitting(false);
    }
  };

  return (
    <section className="glass-card rounded-[32px] p-6">
      <p className="text-xs font-semibold uppercase tracking-[0.24em] text-[var(--accent-dark)]">
        Report an issue
      </p>
      <h2 className="mt-2 text-2xl font-semibold">Send a complaint into the
response queue</h2>
      <form className="mt-6 grid gap-4" onSubmit={handleSubmit}>
        <input
          className="rounded-2xl border border-[var(--border)] bg-white/85 px-4

```

```

py-3 outline-none"
    placeholder="Issue title"
    value={form.title}
    onChange={(event) => updateField("title", event.target.value)}
  />
  <textarea
    className="min-h-32 rounded-2xl border border-[var(--border)] bg-
white/85 px-4 py-3 outline-none"
    placeholder="Describe what happened"
    value={form.description}
    onChange={(event) => updateField("description", event.target.value)}
  />
  <input
    className="rounded-2xl border border-[var(--border)] bg-white/85 px-4
py-3 outline-none"
    placeholder="Location"
    value={form.location}
    onChange={(event) => updateField("location", event.target.value)}
  />
  <button
    className="rounded-full bg-[var(--foreground)] px-5 py-3 text-sm font-
semibold text-white transition hover:bg-[var(--accent-dark)] disabled:opacity-60"
    disabled={submitting}
    type="submit"
  >
    {submitting ? "Submitting..." : "Submit Complaint"}
  </button>
  {message ? <p className="text-sm text-[var(--ink-muted)]">{message}</p> :
null}
</form>
</section>
);
}

```

Web Map Snapshot

File: web_app/src/components/MapComplaint.tsx

```

import type { Complaint } from "../lib/demoData";

type Props = {
  complaints: Complaint[];
};

export default function MapComplaint({ complaints }: Props) {
  return (
    <div className="glass-card rounded-[32px] p-6">
      <div className="mb-5 flex items-center justify-between">
        <div>
          <p className="text-xs font-semibold uppercase tracking-[0.24em] text-

```

```

[var(--ink-muted)]">
    Map Snapshot
  </p>
  <h2 className="mt-2 text-2xl font-semibold">Hotspots by locality</h2>
</div>
<span className="rounded-full bg-white/70 px-3 py-2 text-sm">
  {complaints.length} active markers
</span>
</div>
<div className="relative overflow-hidden rounded-[28px] border border-[var(--border)] bg-[linear-gradient(140deg,#dbeafe,#fef3c7,#fde68a)] p-6">
  <div className="grid gap-4 md:grid-cols-3">
    {complaints.map((complaint, index) => (
      <div
        key={complaint.id}
        className="rounded-3xl bg-white/85 p-4 shadow-sm"
        style={{ transform: `translateY(${index % 2 === 0 ? "0px" :
"10px"})` }}
      >
        <p className="text-xs font-semibold uppercase tracking-[0.22em]
text-[var(--ink-muted)]">
          Zone {index + 1}
        </p>
        <h3 className="mt-2 font-semibold">{complaint.location}</h3>
        <p className="mt-2 text-sm text-[var(--ink-muted)]">
{complaint.title}</p>
      </div>
    ))}
  </div>
</div>
</div>
);
}

```

Web Navbar

File: web_app/src/components/Navbar.tsx

```

import Link from "next/link";

const navLinks = [
  { href: "/dashboard", label: "Dashboard" },
  { href: "/complaints", label: "Complaints" },
  { href: "/admin/dashboard", label: "Admin" },
  { href: "/map", label: "Map" },
  { href: "/profile", label: "Profile" },
];

export default function Navbar() {
  return (

```

```

    <header className="sticky top-0 z-10 px-6 py-5 md:px-10">
      <div className="glass-card mx-auto flex max-w-6xl items-center justify-
between rounded-full px-5 py-3">
        <Link href="/" className="text-lg font-semibold tracking-[0.18em]
uppercase">
          UrbanEye
        </Link>
        <nav className="hidden gap-5 text-sm font-medium text-[var(--ink-muted)]
md:flex">
          {navLinks.map((link) => (
            <Link key={link.href} href={link.href} className="transition
hover:text-[var(--foreground)]">
              {link.label}
            </Link>
          ))}
        </nav>
        <Link
          href="/login"
          className="rounded-full bg-[var(--foreground)] px-4 py-2 text-sm font-
medium text-white transition hover:bg-[var(--accent-dark)]"
        >
          Citizen Login
        </Link>
      </div>
    </header>
  );
}

```

Web Sidebar

File: web_app/src/components/Sidebar.tsx

```

import Link from "next/link";

const links = [
  { href: "/dashboard", label: "Operations Overview" },
  { href: "/complaints", label: "All Complaints" },
  { href: "/admin/dashboard", label: "Admin Console" },
  { href: "/admin/analytics", label: "Analytics" },
];

export default function Sidebar() {
  return (
    <aside className="glass-card rounded-[32px] p-6">
      <p className="text-xs font-semibold uppercase tracking-[0.24em] text-[var(--
ink-muted)]">
        Quick Navigation
      </p>
      <div className="mt-4 flex flex-col gap-3">
        {links.map((link) => (

```

```

        <Link
            key={link.href}
            href={link.href}
            className="rounded-2xl border border-[var(--border)] bg-white/60 px-4
py-3 text-sm font-medium transition hover:-translate-y-0.5 hover:bg-white"
        >
            {link.label}
        </Link>
    )})
</div>
</aside>
);
}

```

Web Complaint Service

File: web_app/src/services/complaintService.ts

```

import { apiRequest } from "../lib/api";
import { demoComplaints, type Complaint } from "../lib/demoData";

export type ComplaintPayload = {
    title: string;
    description: string;
    location: string;
    latitude?: number;
    longitude?: number;
    image?: string;
};

function buildFallbackSocialPost(payload: ComplaintPayload) {
    return `Civic update: ${payload.title} at ${payload.location}.
${payload.description} #CityUpdate #CivicAction`;
}

export async function getComplaints() {
    return apiRequest<Complaint[]>("/complaints", {
        fallbackData: demoComplaints,
    });
}

export async function createComplaint(payload: ComplaintPayload) {
    return apiRequest<Complaint>("/complaints", {
        method: "POST",
        body: JSON.stringify(payload),
        fallbackData: {
            id: `demo-${Date.now()}`,
            title: payload.title,
            description: payload.description,
            location: payload.location,

```

```

        category: "General",
        priority: "MEDIUM",
        status: "PENDING",
        department: "Civic Response Cell",
        createdAt: new Date().toISOString(),
        socialPost: buildFallbackSocialPost(payload),
    },
    });
}

```

Web Admin Service

File: web_app/src/services/adminService.ts

```

import { apiRequest } from "../lib/api";
import { demoOverview, type AdminOverview, type Complaint, type User } from
"../lib/demoData";

export async function getAdminOverview() {
    return apiRequest<AdminOverview>("/admin/overview", {
        fallbackData: demoOverview,
    });
}

export async function getAdminComplaints() {
    return apiRequest<Complaint[]>("/admin/complaints", {
        fallbackData: demoOverview.recentComplaints,
    });
}

export async function getAdminUsers() {
    return apiRequest<User[]>("/admin/users", {
        fallbackData: [
            { name: "Aarav Singh", email: "citizen@urbaneye.dev", role: "CITIZEN" },
            { name: "Meera Rao", email: "authority@urbaneye.dev", role: "AUTHORITY" },
            { name: "Sonal Verma", email: "admin@urbaneye.dev", role: "ADMIN" },
        ],
    });
}

```

Web Demo Data

File: web_app/src/lib/demoData.ts

```

export type User = {
    id?: string;
    name: string;
    email: string;
}

```

```

    role?: string;
};

export type Complaint = {
  id: string;
  title: string;
  description: string;
  location: string;
  category: string;
  priority: string;
  status: string;
  department?: string;
  sentiment?: string;
  confidence?: number;
  suggestedAction?: string;
  createdAt: string;
  socialPost?: string;
  image?: string;
  latitude?: number;
  longitude?: number;
  upvotes?: number;
  user?: User | null;
};

export type AdminOverview = {
  totals: {
    complaints: number;
    pending: number;
    inProgress: number;
    resolved: number;
    users: number;
  };
  statusBreakdown: Record<string, number>;
  priorityBreakdown: Record<string, number>;
  departmentBreakdown: Record<string, number>;
  recentComplaints: Complaint[];
};

export const demoComplaints: Complaint[] = [
  {
    id: "cmp-1001",
    title: "Overflowing roadside garbage",
    description: "Garbage bins near Sector 8 market have not been cleared for three days.",
    location: "Sector 8 Market, Patna",
    category: "Sanitation",
    priority: "HIGH",
    status: "IN_PROGRESS",
    department: "Sanitation Department",
    sentiment: "frustrated",
    confidence: 0.89,
    suggestedAction: "Dispatch sanitation pickup team within 12 hours.",
  }
];

```

```
    createdAt: "2026-03-22T09:30:00.000Z",
    socialPost: "Ticket escalated for sanitation crew update.",
    upvotes: 19,
    latitude: 25.6175,
    longitude: 85.1452,
    user: {
      name: "Aarav Singh",
      email: "citizen@urbaneye.dev",
      role: "CITIZEN",
    },
  },
  {
    id: "cmp-1002",
    title: "Streetlight outage",
    description: "Two consecutive streetlights are out near the school crossing.",
    location: "Boring Road Crossing, Patna",
    category: "Electricity",
    priority: "MEDIUM",
    status: "PENDING",
    department: "Electricity Department",
    sentiment: "concerned",
    confidence: 0.78,
    suggestedAction: "Assign to night maintenance team for inspection.",
    createdAt: "2026-03-21T18:10:00.000Z",
    upvotes: 11,
    latitude: 25.6128,
    longitude: 85.1178,
    user: {
      name: "Aarav Singh",
      email: "citizen@urbaneye.dev",
      role: "CITIZEN",
    },
  },
  {
    id: "cmp-1003",
    title: "Water leakage near bus stop",
    description: "Continuous water leakage is flooding the footpath near the bus stop.",
    location: "Kankarbagh Main Road, Patna",
    category: "Water",
    priority: "HIGH",
    status: "PENDING",
    department: "Water Department",
    sentiment: "urgent",
    confidence: 0.84,
    suggestedAction: "Inspect pipeline pressure and send repair crew.",
    createdAt: "2026-03-23T06:45:00.000Z",
    upvotes: 23,
    latitude: 25.5944,
    longitude: 85.1612,
    user: {
      name: "Aarav Singh",
```

```
    email: "citizen@urbaneye.dev",
    role: "CITIZEN",
  },
},
{
  id: "cmp-1004",
  title: "Pothole causing traffic slowdown",
  description: "A deep pothole has opened up at the flyover approach and bikes
are swerving around it.",
  location: "Bailey Road Flyover, Patna",
  category: "Roads",
  priority: "HIGH",
  status: "RESOLVED",
  department: "Road Department",
  sentiment: "annoyed",
  confidence: 0.92,
  suggestedAction: "Mark area and schedule resurfacing crew.",
  createdAt: "2026-03-19T07:50:00.000Z",
  socialPost: "Resolved after emergency patch work.",
  upvotes: 31,
  latitude: 25.6096,
  longitude: 85.1081,
  user: {
    name: "Aarav Singh",
    email: "citizen@urbaneye.dev",
    role: "CITIZEN",
  },
},
],
];
```

```
export const demoOverview: AdminOverview = {
  totals: {
    complaints: demoComplaints.length,
    pending: 2,
    inProgress: 1,
    resolved: 1,
    users: 3,
  },
  statusBreakdown: {
    PENDING: 2,
    IN_PROGRESS: 1,
    RESOLVED: 1,
  },
  priorityBreakdown: {
    HIGH: 3,
    MEDIUM: 1,
  },
  departmentBreakdown: {
    "Sanitation Department": 1,
    "Electricity Department": 1,
    "Water Department": 1,
    "Road Department": 1,
  },
};
```

```
    },  
    recentComplaints: demoComplaints.slice(0, 3),  
  };  
};
```

Mobile Application Entry

File: mobile_app/App.js

```
// App.js  
import 'react-native-gesture-handler';  
import React from "react";  
import { NavigationContainer } from "@react-navigation/native";  
import AppNavigator from "../src/navigation/AppNavigator";  
import AuthProvider from "../src/context/AuthContext";  
  
export default function App() {  
  return (  
    <AuthProvider>  
      <NavigationContainer>  
        <AppNavigator />  
      </NavigationContainer>  
    </AuthProvider>  
  );  
}
```

Mobile Navigator

File: mobile_app/src/navigation/AppNavigator.js

```
import React, { useContext } from "react";  
import { createNativeStackNavigator } from "@react-navigation/native-stack";  
import { ActivityIndicator, SafeAreaView, StyleSheet, Text, View } from "react-native";  
  
import HomeScreen from "../screens/HomeScreen";  
import ReportScreen from "../screens/ReportScreen";  
import DashboardScreen from "../screens/DashbboardScreen";  
import ComplaintDetailScreen from "../screens/ComplaintDetailScreen";  
import MapScreen from "../screens/map";  
import ProfileScreen from "../screens/ProfileScreen";  
  
import LoginScreen from "../screens/LoginScreen";  
import RegisterScreen from "../screens/RegisterScreen";  
  
import { AuthContext } from "../context/AuthContext";  
  
const Stack = createNativeStackNavigator();
```

```

export default function AppNavigator() {
  const { isHydrated, user } = useContext(AuthContext);

  if (!isHydrated) {
    return (
      <SafeAreaView style={styles.safeArea}>
        <View style={styles.loadingCard}>
          <ActivityIndicator color="#ef8354" size="large" />
          <Text style={styles.loadingTitle}>Loading your civic workspace</Text>
          <Text style={styles.loadingBody}>Restoring profile, complaint tracker,
and saved X account.</Text>
        </View>
      </SafeAreaView>
    );
  }

  return (
    <Stack.Navigator
      screenOptions={{
        headerBackTitle: "Back",
        headerStyle: {
          backgroundColor: "#f4efe6",
        },
        headerShadowVisible: false,
        headerTintColor: "#14213d",
        headerTitleStyle: {
          fontWeight: "700",
        },
      }}
    >
      {user ? (
        <>
          <Stack.Screen name="Home" component={HomeScreen} options={{ title:
"UrbanEye" }} />
          <Stack.Screen name="Report" component={ReportScreen} options={{ title:
"Report Issue" }} />
          <Stack.Screen name="Dashboard" component={DashboardScreen} options={{
title: "Dashboard" }} />
          <Stack.Screen name="Detail" component={ComplaintDetailScreen} options={{
title: "Complaint Detail" }} />
          <Stack.Screen name="Map" component={MapScreen} options={{ title:
"Hotspot Map" }} />
          <Stack.Screen name="Profile" component={ProfileScreen} options={{ title:
"Profile" }} />
        </>
      ) : (
        <>
          <Stack.Screen name="Login" component={LoginScreen} options={{
headerShown: false }} />
          <Stack.Screen name="Register" component={RegisterScreen} options={{
headerShown: false }} />
        </>
      )
    }
  );
}

```

```

    ))
    </Stack.Navigator>
  );
}

const styles = StyleSheet.create({
  safeArea: {
    flex: 1,
    backgroundColor: "#f4efe6",
    justifyContent: "center",
    padding: 24,
  },
  loadingCard: {
    backgroundColor: "#14213d",
    borderRadius: 28,
    padding: 28,
    alignItems: "center",
  },
  loadingTitle: {
    marginTop: 18,
    color: "fff",
    fontSize: 22,
    fontWeight: "800",
    textAlign: "center",
  },
  loadingBody: {
    marginTop: 10,
    color: "#dbe2ef",
    lineHeight: 22,
    textAlign: "center",
  },
});

```

Mobile Auth Context

File: mobile_app/src/context/AuthContext.js

```

import AsyncStorage from "@react-native-async-storage/async-storage";
import React, { createContext, useEffect, useState } from "react";

export const AuthContext = createContext();

const USER_STORAGE_KEY = "urbaneye-user";

export default function AuthProvider({ children }) {
  const [user, setUser] = useState(null);
  const [isHydrated, setIsHydrated] = useState(false);

  useEffect(() => {
    let isMounted = true;

```

```

const hydrateUser = async () => {
  try {
    const storedUser = await AsyncStorage.getItem(USER_STORAGE_KEY);

    if (storedUser && isMounted) {
      setUser(JSON.parse(storedUser));
    }
  } catch (error) {
    if (isMounted) {
      setUser(null);
    }
  } finally {
    if (isMounted) {
      setIsHydrated(true);
    }
  }
};

hydrateUser();

return () => {
  isMounted = false;
};
}, []);

useEffect(() => {
  if (!isHydrated) {
    return;
  }

  const persistUser = async () => {
    if (user) {
      await AsyncStorage.setItem(USER_STORAGE_KEY, JSON.stringify(user));
      return;
    }

    await AsyncStorage.removeItem(USER_STORAGE_KEY);
  };

  persistUser();
}, [isHydrated, user]);

const updateUser = (updates) => {
  setUser((currentUser) => {
    if (!currentUser) {
      return currentUser;
    }

    const nextUpdates =
      typeof updates === "function" ? updates(currentUser) : updates;

```

```

        return {
            ...currentUser,
            ...nextUpdates,
        };
    });
};

return (
    <AuthContext.Provider value={{ isHydrated, updateUser, user, setUser }}>
        {children}
    </AuthContext.Provider>
);
}

```

Mobile Complaint Service

File: mobile_app/src/services/complaintService.js

```

import AsyncStorage from "@react-native-async-storage/async-storage";

import API from "../api";

const LOCAL_COMPLAINTS_KEY = "urbaneye-local-complaints";

function buildFallbackSocialPost(payload) {
    return `Civic update: ${payload.title} at ${payload.location}.
    ${payload.description} #CityUpdate #CivicAction`;
}

async function loadLocalComplaints() {
    try {
        const rawComplaints = await AsyncStorage.getItem(LOCAL_COMPLAINTS_KEY);
        return rawComplaints ? JSON.parse(rawComplaints) : [];
    } catch (error) {
        return [];
    }
}

async function saveLocalComplaints(complaints) {
    try {
        await AsyncStorage.setItem(LOCAL_COMPLAINTS_KEY, JSON.stringify(complaints));
    } catch (error) {
        // Keep the app responsive even if local persistence fails.
    }
}

function mergeComplaints(primaryComplaints, secondaryComplaints = []) {
    const seenIds = new Set();
    const combined = [...primaryComplaints,
        ...secondaryComplaints].filter((complaint) => {

```

```

    if (seenIds.has(complaint.id)) {
        return false;
    }

    seenIds.add(complaint.id);
    return true;
});

return combined.sort((left, right) => new Date(right.createdAt || 0) - new
Date(left.createdAt || 0));
}

async function persistComplaint(complaint) {
    const currentComplaints = await loadLocalComplaints();
    const updatedComplaints = mergeComplaints([complaint], currentComplaints);
    await saveLocalComplaints(updatedComplaints);
}

const demoComplaints = [
    {
        id: "cmp-1001",
        title: "Overflowing roadside garbage",
        description: "Garbage bins near Sector 8 market have not been cleared for
three days.",
        location: "Sector 8 Market, Patna",
        category: "Sanitation",
        priority: "HIGH",
        status: "IN_PROGRESS",
        department: "Sanitation Department",
        suggestedAction: "Dispatch sanitation pickup team within 12 hours.",
        socialPost: "Ticket escalated for sanitation crew update.",
        latitude: 25.6175,
        longitude: 85.1452,
    },
    {
        id: "cmp-1002",
        title: "Streetlight outage",
        description: "Two consecutive streetlights are out near the school crossing.",
        location: "Boring Road Crossing, Patna",
        category: "Electricity",
        priority: "MEDIUM",
        status: "PENDING",
        department: "Electricity Department",
        suggestedAction: "Assign to night maintenance team for inspection.",
        socialPost: "",
        latitude: 25.6128,
        longitude: 85.1178,
    },
    {
        id: "cmp-1003",
        title: "Water leakage near bus stop",
        description: "Continuous water leakage is flooding the footpath near the bus

```

```

stop.",
  location: "Kankarbagh Main Road, Patna",
  category: "Water",
  priority: "HIGH",
  status: "PENDING",
  department: "Water Department",
  suggestedAction: "Inspect pipeline pressure and send repair crew.",
  socialPost: "",
  latitude: 25.5944,
  longitude: 85.1612,
},
];

export const getComplaints = async () => {
  const localComplaints = await loadLocalComplaints();

  try {
    const res = await API.get("/complaints");
    return mergeComplaints(res.data, localComplaints);
  } catch (error) {
    return mergeComplaints(localComplaints, demoComplaints);
  }
};

export const createComplaint = async (payload) => {
  try {
    const res = await API.post("/complaints", payload);
    await persistComplaint(res.data);
    return res.data;
  } catch (error) {
    const complaint = {
      id: `demo-${Date.now()}`,
      title: payload.title,
      description: payload.description,
      location: payload.location,
      category: "General",
      priority: "MEDIUM",
      status: "PENDING",
      department: "Civic Response Cell",
      suggestedAction: "Validate details and route to the appropriate
department.",
      socialPost: buildFallbackSocialPost(payload),
      createdAt: new Date().toISOString(),
    };

    await persistComplaint(complaint);
    return complaint;
  }
};

```

Mobile API Service

File: mobile_app/src/services/api.js

```
import axios from "axios";

const API = axios.create({
  baseURL: process.env.EXPO_PUBLIC_API_URL || "http://192.168.138.51:5000/api",
});

export default API;
```

Mobile Auth Service

File: mobile_app/src/services/authService.js

```
import API from "../api";

function buildDemoUser(data) {
  return {
    id: `demo-${Date.now()}`,
    name: data.name || "Demo Citizen",
    email: data.email,
    role: "CITIZEN",
    createdAt: new Date().toISOString(),
  };
}

export const loginUser = async (data) => {
  try {
    const res = await API.post("/auth/login", data);
    return res.data;
  } catch (error) {
    if (!error.response) {
      return { user: buildDemoUser(data) };
    }

    throw error;
  }
};

export const registerUser = async (data) => {
  try {
    const res = await API.post("/auth/register", data);
    return res.data;
  } catch (error) {
    if (!error.response) {
      return { user: buildDemoUser(data) };
    }
  }
}
```

```
        throw error;
    }
};
```

Mobile Home Screen

File: mobile_app/src/screens/HomeScreen.js

```
import React, { useContext, useEffect, useState } from "react";
import { SafeAreaView, ScrollView, StyleSheet, Text, TouchableOpacity, View } from
"react-native";

import { AuthContext } from "../context/AuthContext";
import { getComplaints } from "../services/complaintService";
import { getComplaintStats } from "../utils/complaintUtils";

const actions = [
  {
    label: "Report a new issue",
    hint: "Capture a fresh complaint with location and photo evidence.",
    screen: "Report",
    tone: "#14213d",
  },
  {
    label: "Open complaint dashboard",
    hint: "See which complaints are solved, in progress, or still waiting.",
    screen: "Dashboard",
    tone: "#ef8354",
  },
  {
    label: "Review hotspot map",
    hint: "Scan city trouble zones and active civic clusters.",
    screen: "Map",
    tone: "#2a9d8f",
  },
  {
    label: "Manage your profile",
    hint: "Update resident details and connect your X account.",
    screen: "Profile",
    tone: "#6c7aa1",
  },
];

export default function HomeScreen({ navigation }) {
  const { user } = useContext(AuthContext);
  const [complaints, setComplaints] = useState([]);

  useEffect(() => {
    let isMounted = true;
```

```

const loadComplaints = async () => {
  const response = await getComplaints();

  if (isMounted) {
    setComplaints(response);
  }
};

loadComplaints();
const unsubscribe = navigation.addListener("focus", loadComplaints);

return () => {
  isMounted = false;
  unsubscribe();
};
}, [navigation]));

const stats = getComplaintStats(complaints);

return (
  <SafeAreaView style={styles.safeArea}>
    <ScrollView contentContainerStyle={styles.content}
showsVerticalScrollIndicator={false}>
      <View style={styles.hero}>
        <Text style={styles.eyebrow}>UrbanEye citizen app</Text>
        <Text style={styles.title}>Stay on top of every civic complaint</Text>
        <Text style={styles.subtitle}>
          Report issues, monitor whether they are solved yet, and keep your
public update channel ready.
        </Text>

        <View style={styles.heroBadgeRow}>
          <View style={styles.heroBadge}>
            <Text style={styles.heroBadgeValue}>{stats.resolved}</Text>
            <Text style={styles.heroBadgeLabel}>Solved</Text>
          </View>
          <View style={styles.heroBadge}>
            <Text style={styles.heroBadgeValue}>{stats.pending +
stats.inProgress}</Text>
            <Text style={styles.heroBadgeLabel}>Open</Text>
          </View>
          <View style={styles.heroBadge}>
            <Text style={styles.heroBadgeValue}>{user?.xHandle ? "Yes" : "Add"}
</Text>
            <Text style={styles.heroBadgeLabel}>X linked</Text>
          </View>
        </View>

        <View style={styles.section}>
          <Text style={styles.sectionTitle}>Quick status snapshot</Text>

```

```

<View style={styles.statGrid}>
  <View style={styles.statCard}>
    <Text style={styles.statValue}>{stats.total}</Text>
    <Text style={styles.statLabel}>Complaints tracked</Text>
  </View>
  <View style={[styles.statCard, styles.statCardWarm]}>
    <Text style={styles.statValue}>{stats.pending}</Text>
    <Text style={styles.statLabel}>Waiting for action</Text>
  </View>
  <View style={[styles.statCard, styles.statCardCool]}>
    <Text style={styles.statValue}>{stats.inProgress}</Text>
    <Text style={styles.statLabel}>In progress</Text>
  </View>
</View>
</View>
</View>

<View style={styles.section}>
  <Text style={styles.sectionTitle}>Quick actions</Text>
  <View style={styles.actionList}>
    {actions.map((action) => (
      <TouchableOpacity
        key={action.screen}
        style={[styles.actionCard, { borderLeftColor: action.tone }]}
        onPress={() => navigation.navigate(action.screen)}
      >
        <Text style={styles.actionLabel}>{action.label}</Text>
        <Text style={styles.actionHint}>{action.hint}</Text>
        <Text style={styles.actionCta}>Open</Text>
      </TouchableOpacity>
    ))}
  </View>
</View>

<View style={styles.socialCard}>
  <Text style={styles.sectionTitle}>X account connection</Text>
  <Text style={styles.socialBody}>
    {user?.xHandle
      ? `Connected as @${user.xHandle}. Your complaint updates can now
reference your preferred X handle.`
      : "Add your X handle in Profile so complaint-related posts and
updates can be prepared around your public account."}
  </Text>
  <TouchableOpacity style={styles.socialButton} onPress={() =>
navigation.navigate("Profile")}>
    <Text style={styles.socialButtonText}>{user?.xHandle ? "Manage X
account" : "Add X account"}</Text>
  </TouchableOpacity>
</View>
</ScrollView>
</SafeAreaView>
);
}

```

```
const styles = StyleSheet.create({
  safeArea: {
    flex: 1,
    backgroundColor: "#f4efe6",
  },
  content: {
    padding: 20,
    gap: 24,
  },
  hero: {
    backgroundColor: "#14213d",
    borderRadius: 28,
    padding: 24,
    overflow: "hidden",
  },
  eyebrow: {
    color: "#ef8354",
    fontSize: 12,
    fontWeight: "700",
    letterSpacing: 2,
    textTransform: "uppercase",
  },
  title: {
    color: "#fff",
    fontSize: 32,
    fontWeight: "800",
    marginTop: 10,
  },
  subtitle: {
    color: "#dbe2ef",
    fontSize: 16,
    lineHeight: 24,
    marginTop: 12,
  },
  heroBadgeRow: {
    flexDirection: "row",
    gap: 10,
    marginTop: 22,
  },
  heroBadge: {
    flex: 1,
    backgroundColor: "rgba(255,255,255,0.12)",
    borderRadius: 20,
    paddingVertical: 14,
    paddingHorizontal: 12,
  },
  heroBadgeValue: {
    color: "#fff",
    fontSize: 20,
    fontWeight: "800",
  },
},
```

```
heroBadgeLabel: {
  marginTop: 6,
  color: "#dbe2ef",
  fontSize: 12,
  fontWeight: "600",
},
section: {
  gap: 14,
},
sectionTitle: {
  color: "#14213d",
  fontSize: 22,
  fontWeight: "800",
},
statGrid: {
  flexDirection: "row",
  flexWrap: "wrap",
  gap: 12,
},
statCard: {
  flex: 1,
  minWidth: "30%",
  backgroundColor: "#fff",
  borderRadius: 24,
  padding: 18,
  shadowColor: "#14213d",
  shadowOpacity: 0.08,
  shadowRadius: 12,
  shadowOffset: { width: 0, height: 8 },
  elevation: 2,
},
statCardWarm: {
  backgroundColor: "#fff7eb",
},
statCardCool: {
  backgroundColor: "#edf6ff",
},
statValue: {
  color: "#14213d",
  fontSize: 26,
  fontWeight: "800",
},
statLabel: {
  marginTop: 8,
  color: "#5c677d",
  lineHeight: 20,
},
actionList: {
  gap: 14,
},
actionCard: {
  backgroundColor: "#fff",
```

```
borderRadius: 22,
padding: 18,
borderLeftWidth: 6,
shadowColor: "#14213d",
shadowOpacity: 0.08,
shadowRadius: 12,
shadowOffset: { width: 0, height: 8 },
elevation: 2,
},
actionLabel: {
  color: "#14213d",
  fontSize: 18,
  fontWeight: "700",
},
actionHint: {
  marginTop: 8,
  color: "#5c677d",
  lineHeight: 21,
},
actionCta: {
  marginTop: 14,
  color: "#14213d",
  fontWeight: "700",
},
socialCard: {
  backgroundColor: "#fff",
  borderRadius: 28,
  padding: 20,
  borderWidth: 1,
  borderColor: "#e7dccc",
},
socialBody: {
  marginTop: 10,
  color: "#5c677d",
  lineHeight: 22,
},
socialButton: {
  marginTop: 18,
  alignSelf: "flex-start",
  backgroundColor: "#ef8354",
  paddingHorizontal: 18,
  paddingVertical: 12,
  borderRadius: 999,
},
socialButtonText: {
  color: "#fff",
  fontSize: 14,
  fontWeight: "700",
},
});
```

Mobile Dashboard Screen

File: mobile_app/src/screens/DashbboardScreen.js

```
import React, { useEffect, useMemo, useState } from "react";
import {
  FlatList,
  RefreshControl,
  SafeAreaView,
  StyleSheet,
  Text,
  TouchableOpacity,
  View,
} from "react-native";

import { getComplaints } from "../services/complaintService";
import {
  DASHBOARD_FILTERS,
  filterComplaints,
  getComplaintStats,
} from "../utils/complaintUtils";
import ComplaintCard from "../ComplaintCard";

export default function DashboardScreen({ navigation }) {
  const [data, setData] = useState([]);
  const [refreshing, setRefreshing] = useState(false);
  const [activeFilter, setActiveFilter] = useState("ALL");

  useEffect(() => {
    let isMounted = true;

    const loadData = async () => {
      const response = await getComplaints();

      if (isMounted) {
        setData(response);
      }
    };

    loadData();
    const unsubscribe = navigation.addListener("focus", loadData);

    return () => {
      isMounted = false;
      unsubscribe();
    };
  }, [navigation]);

  const onRefresh = async () => {
    setRefreshing(true);
    const response = await getComplaints();
```

```

    setData(response);
    setRefreshing(false);
  };

  const stats = useMemo(() => getComplaintStats(data), [data]);
  const filteredData = useMemo(() => filterComplaints(data, activeFilter),
[activeFilter, data]);
  const solvedRate = stats.total ? Math.round((stats.resolved / stats.total) *
100) : 0;
  const openCount = stats.pending + stats.inProgress;

  return (
    <SafeAreaView style={styles.safeArea}>
      <FlatList
        keyExtractor={(item) => item.id.toString()}
        contentContainerStyle={styles.content}
        refreshControl={<RefreshControl refreshing={refreshing} onRefresh=
{onRefresh} />}
        ListHeaderComponent={
          <>
            <View style={styles.hero}>
              <Text style={styles.eyebrow}>Complaint tracker</Text>
              <Text style={styles.title}>See what is solved and what still needs
attention</Text>
              <Text style={styles.subtitle}>
                Follow every complaint through pending, in-progress, and solved
states from one place.
              </Text>

              <View style={styles.progressShell}>
                <View style={styles.progressHeader}>
                  <Text style={styles.progressTitle}>Resolution progress</Text>
                  <Text style={styles.progressValue}>{solvedRate}% solved</Text>
                </View>
                <View style={styles.progressTrack}>
                  <View style={[styles.progressFill, { width: `${solvedRate}%` }]}
/>

                  </View>
                  <Text style={styles.progressCopy}>
                    {stats.resolved} solved and {openCount} still open.
                  </Text>
                </View>
              </View>

              <View style={styles.summaryRow}>
                <View style={styles.summaryCard}>
                  <Text style={styles.summaryValue}>{stats.total}</Text>
                  <Text style={styles.summaryLabel}>Tracked</Text>
                </View>
                <View style={[styles.summaryCard, styles.summaryCardWarm]}>
                  <Text style={styles.summaryValue}>{stats.pending}</Text>
                  <Text style={styles.summaryLabel}>Pending</Text>

```

```

        </View>
        <View style={[styles.summaryCard, styles.summaryCardCool]}>
            <Text style={styles.summaryValue}>{stats.inProgress}</Text>
            <Text style={styles.summaryLabel}>In progress</Text>
        </View>
        <View style={[styles.summaryCard, styles.summaryCardSuccess]}>
            <Text style={styles.summaryValue}>{stats.resolved}</Text>
            <Text style={styles.summaryLabel}>Solved</Text>
        </View>
    </View>

    <View style={styles.filtersCard}>
        <Text style={styles.filtersTitle}>Filter by status</Text>
        <View style={styles.filterRow}>
            {DASHBOARD_FILTERS.map((filter) => {
                const isActive = activeFilter === filter.key;

                return (
                    <TouchableOpacity
                        key={filter.key}
                        style={[styles.filterChip, isActive &&
styles.filterChipActive]}
                        onPress={() => setActiveFilter(filter.key)}
                    >
                        <Text style={[styles.filterLabel, isActive &&
styles.filterLabelActive]}>
                            {filter.label}
                        </Text>
                    </TouchableOpacity>
                );
            })}
        </View>
    </View>
</>
}

ListEmptyComponent={
    <View style={styles.emptyCard}>
        <Text style={styles.emptyTitle}>No complaints in this view</Text>
        <Text style={styles.emptyBody}>
            Switch filters or submit a new complaint to start tracking its
resolution status.
        </Text>
    </View>
}

renderItem={({ item }) => <ComplaintCard item={item} navigation=
{navigation} />}
    data={filteredData}
  />
</SafeAreaView>
);
}

```

```
const styles = StyleSheet.create({
  safeArea: {
    flex: 1,
    backgroundColor: "#f4efe6",
  },
  content: {
    padding: 16,
    gap: 14,
  },
  hero: {
    backgroundColor: "#14213d",
    borderRadius: 26,
    padding: 22,
  },
  eyebrow: {
    color: "#ef8354",
    fontSize: 12,
    fontWeight: "700",
    letterSpacing: 2,
    textTransform: "uppercase",
  },
  title: {
    marginTop: 8,
    fontSize: 28,
    fontWeight: "800",
    color: "#fff",
  },
  subtitle: {
    marginTop: 10,
    color: "#dbe2ef",
    lineHeight: 22,
  },
  progressShell: {
    marginTop: 22,
    backgroundColor: "rgba(255,255,255,0.12)",
    borderRadius: 22,
    padding: 16,
  },
  progressHeader: {
    flexDirection: "row",
    justifyContent: "space-between",
    gap: 12,
  },
  progressTitle: {
    color: "#fff",
    fontSize: 14,
    fontWeight: "700",
  },
  progressValue: {
    color: "#efc17b",
    fontSize: 14,
    fontWeight: "700",
  },
});
```

```
    },
    progressTrack: {
      height: 10,
      borderRadius: 999,
      backgroundColor: "rgba(255,255,255,0.16)",
      overflow: "hidden",
      marginTop: 12,
    },
    progressFill: {
      height: "100%",
      borderRadius: 999,
      backgroundColor: "#2a9d8f",
    },
    progressCopy: {
      marginTop: 10,
      color: "#dbe2ef",
      lineHeight: 20,
    },
    summaryRow: {
      flexDirection: "row",
      flexWrap: "wrap",
      gap: 10,
      marginTop: 14,
    },
    summaryCard: {
      flex: 1,
      minWidth: "47%",
      backgroundColor: "#fff",
      borderRadius: 22,
      padding: 18,
    },
    summaryCardWarm: {
      backgroundColor: "#fff7eb",
    },
    summaryCardCool: {
      backgroundColor: "#edf6ff",
    },
    summaryCardSuccess: {
      backgroundColor: "#edfdf4",
    },
    summaryValue: {
      color: "#14213d",
      fontSize: 24,
      fontWeight: "800",
    },
    summaryLabel: {
      marginTop: 8,
      color: "#5c677d",
      fontWeight: "600",
    },
    filtersCard: {
      backgroundColor: "#fff",
```

```

        borderRadius: 24,
        padding: 18,
        marginTop: 14,
      },
      filtersTitle: {
        color: "#14213d",
        fontSize: 18,
        fontWeight: "800",
      },
      filterRow: {
        flexDirection: "row",
        flexWrap: "wrap",
        gap: 10,
        marginTop: 14,
      },
      filterChip: {
        borderRadius: 999,
        backgroundColor: "#f3efe7",
        paddingHorizontal: 14,
        paddingVertical: 10,
      },
      filterChipActive: {
        backgroundColor: "#14213d",
      },
      filterLabel: {
        color: "#5c677d",
        fontWeight: "700",
      },
      filterLabelActive: {
        color: "#fff",
      },
      emptyCard: {
        backgroundColor: "#fff",
        borderRadius: 22,
        padding: 22,
        marginTop: 8,
      },
      emptyTitle: {
        color: "#14213d",
        fontSize: 18,
        fontWeight: "800",
      },
      emptyBody: {
        marginTop: 8,
        color: "#5c677d",
        lineHeight: 22,
      },
    }
  ));

```

Mobile Profile Screen

File: mobile_app/src/screens/ProfileScreen.js

```
import React, { useContext, useEffect, useState } from "react";
import {
  SafeAreaView,
  ScrollView,
  StyleSheet,
  Text,
  TextInput,
  TouchableOpacity,
  View,
} from "react-native";

import { AuthContext } from "../../context/AuthContext";

export default function ProfileScreen() {
  const { updateUser, user, setUser } = useContext(AuthContext);
  const [xHandle, setXHandle] = useState(user?.xHandle || "");
  const [message, setMessage] = useState("");

  useEffect(() => {
    setXHandle(user?.xHandle || "");
  }, [user?.xHandle]);

  const handleSaveXAccount = () => {
    const normalizedHandle = xHandle.replace(/^@+/, "").trim();

    if (!normalizedHandle) {
      setMessage("Enter an X username to save the account.");
      return;
    }

    if (!/^[A-Za-z0-9_]{1,15}$/.test(normalizedHandle)) {
      setMessage("Use a valid X handle with letters, numbers, or underscores only.");
      return;
    }

    updateUser({ xHandle: normalizedHandle });
    setXHandle(normalizedHandle);
    setMessage(`X account saved as @${normalizedHandle}.`);
  };

  const handleRemoveXAccount = () => {
    updateUser({ xHandle: "" });
    setXHandle("");
    setMessage("Saved X account removed.");
  };

  return (
    <SafeAreaView style={styles.safeArea}>
```

```

<ScrollView contentContainerStyle={styles.container}
showsVerticalScrollIndicator={false}>
  <View style={styles.hero}>
    <Text style={styles.eyebrow}>Resident profile</Text>
    <Text style={styles.name}>{user?.name || "Demo Citizen"}</Text>
    <Text style={styles.email}>{user?.email || "citizen@urbaneye.dev"}
  </Text>

  <View style={styles.heroMetaRow}>
    <View style={styles.heroMetaCard}>
      <Text style={styles.heroMetaValue}>{user?.role || "CITIZEN"}</Text>
      <Text style={styles.heroMetaLabel}>Role</Text>
    </View>
    <View style={styles.heroMetaCard}>
      <Text style={styles.heroMetaValue}>{user?.xHandle ? "Connected" :
"Not added"}</Text>
      <Text style={styles.heroMetaLabel}>X account</Text>
    </View>
  </View>
  <View style={styles.card}>
    <Text style={styles.cardLabel}>Connected X account</Text>
    <Text style={styles.cardDescription}>
      Add your X handle to prepare complaint-related updates around the
account you want to use publicly.
    </Text>

    <View style={styles.handleInputShell}>
      <Text style={styles.handlePrefix}>@</Text>
      <TextInput
        autoCapitalize="none"
        autoCorrect={false}
        placeholder="your_handle"
        placeholderTextColor="#9aa4b2"
        style={styles.handleInput}
        value={xHandle}
        onChangeText={(value) => {
          setMessage("");
          setXHandle(value.replace(/^@+/, ""));
        }}
      />
    </View>

    <TouchableOpacity style={styles.primaryButton} onPress=
{handleSaveXAccount}>
      <Text style={styles.primaryButtonText}>{user?.xHandle ? "Update X
account" : "Save X account"}</Text>
    </TouchableOpacity>

    {user?.xHandle ? (
      <TouchableOpacity style={styles.secondaryButton} onPress=

```

```

{handleRemoveXAccount}>
    <Text style={styles.secondaryButtonText}>Remove saved X
account</Text>
    </TouchableOpacity>
    ) : null}

    <Text style={styles.connectedText}>
        {user?.xHandle ? `Currently connected: @${user.xHandle}` : "No X
account saved yet."}
    </Text>
    {message ? <Text style={styles.message}>{message}</Text> : null}
</View>

<View style={styles.card}>
    <Text style={styles.cardLabel}>Account summary</Text>
    <Text style={styles.cardValue}>{user?.role || "CITIZEN"}</Text>
    <Text style={styles.cardDescription}>
        Use the dashboard to check whether your complaints are solved, in
progress, or still waiting for action.
    </Text>
</View>

    <TouchableOpacity style={styles.logoutButton} onPress={() =>
setUser(null)}>
        <Text style={styles.logoutText}>Logout</Text>
    </TouchableOpacity>
</ScrollView>
</SafeAreaView>
);
}

const styles = StyleSheet.create({
  safeArea: {
    flex: 1,
    backgroundColor: "#f4efe6",
  },
  container: {
    padding: 20,
    gap: 18,
  },
  hero: {
    backgroundColor: "#14213d",
    borderRadius: 28,
    padding: 24,
  },
  eyebrow: {
    color: "#ef8354",
    fontSize: 12,
    fontWeight: "700",
    letterSpacing: 2,
    textTransform: "uppercase",
  },

```

```
name: {
  color: "#fff",
  fontSize: 28,
  fontWeight: "800",
  marginTop: 10,
},
email: {
  color: "#dbe2ef",
  marginTop: 8,
},
heroMetaRow: {
  flexDirection: "row",
  gap: 12,
  marginTop: 20,
},
heroMetaCard: {
  flex: 1,
  backgroundColor: "rgba(255,255,255,0.12)",
  borderRadius: 20,
  padding: 14,
},
heroMetaValue: {
  color: "#fff",
  fontWeight: "800",
  fontSize: 16,
},
heroMetaLabel: {
  color: "#dbe2ef",
  fontSize: 12,
  fontWeight: "600",
  marginTop: 6,
},
card: {
  backgroundColor: "#fff",
  borderRadius: 24,
  padding: 18,
  borderWidth: 1,
  borderColor: "#e7dccc",
},
cardLabel: {
  color: "#6b7280",
  fontSize: 13,
  textTransform: "uppercase",
  letterSpacing: 1.2,
  fontWeight: "700",
},
cardValue: {
  color: "#14213d",
  fontSize: 22,
  fontWeight: "700",
  marginTop: 8,
},
```

```
cardDescription: {
  color: "#5c677d",
  lineHeight: 22,
  marginTop: 10,
},
handleInputShell: {
  flexDirection: "row",
  alignItems: "center",
  marginTop: 18,
  borderWidth: 1,
  borderColor: "#e5e7eb",
  borderRadius: 18,
  backgroundColor: "#fcfaf6",
},
handlePrefix: {
  paddingLeft: 16,
  color: "#14213d",
  fontSize: 18,
  fontWeight: "800",
},
handleInput: {
  flex: 1,
  paddingVertical: 14,
  paddingHorizontal: 10,
  color: "#14213d",
},
primaryButton: {
  marginTop: 16,
  backgroundColor: "#0f172a",
  borderRadius: 999,
  paddingVertical: 15,
  alignItems: "center",
},
primaryButtonText: {
  color: "#fff",
  fontWeight: "700",
  fontSize: 15,
},
secondaryButton: {
  marginTop: 10,
  borderRadius: 999,
  borderWidth: 1,
  borderColor: "#d8dde6",
  paddingVertical: 14,
  alignItems: "center",
},
secondaryButtonText: {
  color: "#4f5d75",
  fontWeight: "700",
},
connectedText: {
  marginTop: 14,
```

```

        color: "#14213d",
        fontWeight: "700",
    },
    message: {
        marginTop: 8,
        color: "#5c677d",
        lineHeight: 20,
    },
    logoutButton: {
        backgroundColor: "#ef8354",
        paddingVertical: 16,
        borderRadius: 999,
        alignItems: "center",
    },
    logoutText: {
        color: "#fff",
        fontWeight: "700",
        fontSize: 16,
    },
  },
});

```

Mobile Report Screen

File: mobile_app/src/screens/ReportScreen.js

```

import React, { useState } from "react";
import {
  Alert,
  Image,
  SafeAreaView,
  ScrollView,
  StyleSheet,
  Text,
  TextInput,
  TouchableOpacity,
  View,
} from "react-native";
import * as ImagePicker from "expo-image-picker";

import { createComplaint } from "../services/complaintService";

export default function ReportScreen({ navigation }) {
  const [title, setTitle] = useState("");
  const [description, setDescription] = useState("");
  const [location, setLocation] = useState("");
  const [image, setImage] = useState(null);

  const pickImage = async () => {
    const result = await ImagePicker.launchImageLibraryAsync({
      mediaTypes: ["images"],

```

```

        quality: 0.7,
    });

    if (!result.canceled) {
        setImage(result.assets[0].uri);
    }
};

const handleSubmit = async () => {
    if (!title || !description || !location) {
        Alert.alert("Missing details", "Please complete title, description, and
location.");
        return;
    }

    try {
        await createComplaint({
            title,
            description,
            location,
            image,
        });

        Alert.alert("Success", "Complaint submitted");
        setTitle("");
        setDescription("");
        setLocation("");
        setImage(null);
        navigation.navigate("Dashboard");
    } catch (error) {
        Alert.alert("Error", "Failed to submit complaint");
    }
};

return (
    <SafeAreaView style={styles.safeArea}>
        <ScrollView contentContainerStyle={styles.content}>
            <View style={styles.hero}>
                <Text style={styles.eyebrow}>Report issue</Text>
                <Text style={styles.title}>Send a civic complaint for AI triage</Text>
            </View>

            <View style={styles.form}>
                <TextInput
                    placeholder="Issue title"
                    placeholderTextColor="#8d99ae"
                    style={styles.input}
                    value={title}
                    onChangeText={setTitle}
                />
                <TextInput
                    placeholder="Describe the issue"

```

```

        placeholderTextColor="#8d99ae"
        style={styles.input, styles.textarea]}
        value={description}
        onChangeText={setDescription}
        multiline
    />
    <TextInput
        placeholder="Location"
        placeholderTextColor="#8d99ae"
        style={styles.input}
        value={location}
        onChangeText={setLocation}
    />

    <TouchableOpacity style={styles.uploadButton} onPress={pickImage}>
        <Text style={styles.uploadText}>{image ? "Change image" : "Upload
image"}</Text>
    </TouchableOpacity>

    {image ? <Image source={{ uri: image }} style={styles.preview} /> :
null}

    <TouchableOpacity style={styles.submitButton} onPress={handleSubmit}>
        <Text style={styles.submitText}>Submit Complaint</Text>
    </TouchableOpacity>
</View>
</ScrollView>
</SafeAreaView>
);
}

const styles = StyleSheet.create({
  safeArea: {
    flex: 1,
    backgroundColor: "#f4efe6",
  },
  content: {
    padding: 20,
    gap: 18,
  },
  hero: {
    backgroundColor: "#14213d",
    borderRadius: 28,
    padding: 24,
  },
  eyebrow: {
    color: "#ef8354",
    fontSize: 12,
    fontWeight: "700",
    textTransform: "uppercase",
    letterSpacing: 2,
  },

```

```
title: {
  color: "#fff",
  marginTop: 10,
  fontSize: 28,
  fontWeight: "800",
},
form: {
  backgroundColor: "#fff",
  borderRadius: 28,
  padding: 20,
  gap: 12,
},
input: {
  borderWidth: 1,
  borderColor: "#e5e7eb",
  padding: 14,
  borderRadius: 16,
  color: "#14213d",
},
textarea: {
  minHeight: 120,
  textAlignVertical: "top",
},
uploadButton: {
  borderRadius: 999,
  borderWidth: 1,
  borderColor: "#14213d",
  paddingVertical: 14,
  alignItems: "center",
},
uploadText: {
  color: "#14213d",
  fontWeight: "700",
},
preview: {
  width: "100%",
  height: 220,
  borderRadius: 24,
},
submitButton: {
  backgroundColor: "#ef8354",
  borderRadius: 999,
  paddingVertical: 16,
  alignItems: "center",
},
submitText: {
  color: "#fff",
  fontWeight: "700",
  fontSize: 16,
},
});
```

Mobile Complaint Card

File: mobile_app/src/screens/ComplaintCard.js

```
import React from "react";
import { Text, StyleSheet, TouchableOpacity, View } from "react-native";

import { getPriorityMeta, getStatusMeta } from "../utils/complaintUtils";

export default function ComplaintCard({ item, navigation }) {
  const statusMeta = getStatusMeta(item.status);
  const priorityMeta = getPriorityMeta(item.priority);

  return (
    <TouchableOpacity style={styles.card} onPress={() =>
navigation.navigate("Detail", { item })}>
      <View style={styles.topRow}>
        <Text style={styles.category}>{item.category}</Text>
        <View
          style={[
            styles.statusPill,
            {
              backgroundColor: statusMeta.backgroundColor,
              borderColor: statusMeta.borderColor,
            },
          ]>
          <Text style={[styles.statusText, { color: statusMeta.textColor }]}>
{statusMeta.label}</Text>
        </View>
      </View>

      <Text style={styles.title}>{item.title || item.description}</Text>
      <Text numberOfLines={2} style={styles.description}>
        {item.description}
      </Text>

      <View style={styles.metaRow}>
        <Text
          style={[
            styles.meta,
            {
              backgroundColor: priorityMeta.backgroundColor,
              color: priorityMeta.textColor,
            },
          ]>
          <Priority {priorityMeta.label}>
        </Text>
        <Text style={styles.meta}>{item.department}</Text>
      </View>
    </TouchableOpacity>
  );
}
```

```

        <Text style={styles.location}>{item.location}</Text>

        <View style={styles.footerRow}>
            <Text style={styles.footerLabel}>{statusMeta.shortLabel}</Text>
            <Text style={styles.footerHint}>Open details</Text>
        </View>
    </TouchableOpacity>
  );
}

const styles = StyleSheet.create({
  card: {
    backgroundColor: "#fff",
    padding: 18,
    borderRadius: 22,
    borderWidth: 1,
    borderColor: "#ece3d6",
    shadowColor: "#14213d",
    shadowOpacity: 0.08,
    shadowRadius: 12,
    shadowOffset: { width: 0, height: 8 },
    elevation: 2,
  },
  topRow: {
    flexDirection: "row",
    justifyContent: "space-between",
    alignItems: "center",
    gap: 10,
  },
  category: {
    color: "#ef8354",
    fontSize: 12,
    fontWeight: "700",
    letterSpacing: 1.5,
    textTransform: "uppercase",
  },
  statusPill: {
    borderRadius: 999,
    borderWidth: 1,
    paddingHorizontal: 12,
    paddingVertical: 7,
  },
  statusText: {
    fontSize: 12,
    fontWeight: "700",
  },
  title: {
    marginTop: 10,
    color: "#14213d",
    fontSize: 18,
    fontWeight: "800",
  },

```

```

    },
    description: {
      marginTop: 8,
      color: "#5c677d",
      lineHeight: 21,
    },
    metaRow: {
      flexDirection: "row",
      flexWrap: "wrap",
      gap: 8,
      marginTop: 12,
    },
    meta: {
      backgroundColor: "#eef2f7",
      color: "#14213d",
      paddingHorizontal: 10,
      paddingVertical: 6,
      borderRadius: 999,
      overflow: "hidden",
      fontSize: 12,
      fontWeight: "600",
    },
    location: {
      marginTop: 12,
      color: "#6b7280",
      fontSize: 13,
    },
    footerRow: {
      marginTop: 16,
      paddingTop: 14,
      borderTopWidth: 1,
      borderTopColor: "#f1e2e1",
      flexDirection: "row",
      justifyContent: "space-between",
      alignItems: "center",
      gap: 12,
    },
    footerLabel: {
      color: "#14213d",
      fontWeight: "700",
    },
    footerHint: {
      color: "#7b8794",
      fontSize: 12,
      fontWeight: "600",
    },
  },
});

```

Mobile Complaint Detail Screen

File: mobile_app/src/screens/ComplaintDetailScreen.js

```

import React from "react";
import { Image, SafeAreaView, ScrollView, StyleSheet, Text, View } from "react-native";

import { getPriorityMeta, getResolutionMessage, getStatusMeta } from
"../utils/complaintUtils";

export default function ComplaintDetailScreen({ route }) {
  const { item } = route.params;
  const statusMeta = getStatusMeta(item.status);
  const priorityMeta = getPriorityMeta(item.priority);

  return (
    <SafeAreaView style={styles.safeArea}>
      <ScrollView contentContainerStyle={styles.content}>
        <View style={styles.hero}>
          <View
            style={[
              styles.statusPill,
              {
                backgroundColor: statusMeta.backgroundColor,
                borderColor: statusMeta.borderColor,
              },
            ]>
            <Text style={[styles.statusText, { color: statusMeta.textColor }]}>
{statusMeta.label}</Text>
          </View>
          <Text style={styles.category}><item.category></Text>
          <Text style={styles.title}><item.title || item.description></Text>
          <Text style={styles.description}><item.description></Text>
        </View>

        <View style={styles.section}>
          <Text style={styles.sectionTitle}>Resolution status</Text>
          <Text style={styles.resolutionBody}><getResolutionMessage(item.status)>
</Text>

          <View style={styles.detailGrid}>
            <View style={styles.detailCard}>
              <Text style={styles.detailLabel}>Priority</Text>
              <Text
                style={[
                  styles.detailValue,
                  { color: priorityMeta.textColor },
                ]>
                <priorityMeta.label>
              </Text>
            </View>
            <View style={styles.detailCard}>
              <Text style={styles.detailLabel}>Department</Text>

```

```

        <Text style={styles.detailValue}>{item.department || "Civic Response
Cell"}</Text>
    </View>
    <View style={styles.detailCard}>
        <Text style={styles.detailLabel}>Location</Text>
        <Text style={styles.detailValue}>{item.location}</Text>
    </View>
    <View style={styles.detailCard}>
        <Text style={styles.detailLabel}>Current state</Text>
        <Text style={styles.detailValue}>{statusMeta.shortLabel}</Text>
    </View>
</View>
</View>

{item.suggestedAction ? (
    <View style={styles.section}>
        <Text style={styles.sectionTitle}>Suggested Action</Text>
        <Text style={styles.description}>{item.suggestedAction}</Text>
    </View>
) : null}

{item.image ? <Image source={{ uri: item.image }} style={styles.image} />
: null}

{item.socialPost ? (
    <View style={styles.section}>
        <Text style={styles.sectionTitle}>Social Update</Text>
        <Text style={styles.description}>{item.socialPost}</Text>
    </View>
) : null}
</ScrollView>
</SafeAreaView>
);
}

const styles = StyleSheet.create({
  safeArea: {
    flex: 1,
    backgroundColor: "#f4efe6",
  },
  content: {
    padding: 16,
    gap: 16,
  },
  hero: {
    backgroundColor: "#14213d",
    borderRadius: 28,
    padding: 22,
  },
  statusPill: {
    alignSelf: "flex-start",
    borderRadius: 999,

```

```
    borderWidth: 1,
    paddingHorizontal: 12,
    paddingVertical: 7,
  },
  statusText: {
    fontSize: 12,
    fontWeight: "800",
  },
  category: {
    color: "#ef8354",
    fontSize: 12,
    fontWeight: "700",
    letterSpacing: 2,
    textTransform: "uppercase",
    marginTop: 14,
  },
  title: {
    color: "#fff",
    marginTop: 10,
    fontSize: 26,
    fontWeight: "800",
  },
  description: {
    marginTop: 10,
    color: "#dbe2ef",
    lineHeight: 22,
  },
  section: {
    backgroundColor: "#fff",
    borderRadius: 24,
    padding: 18,
  },
  sectionTitle: {
    color: "#14213d",
    fontSize: 18,
    fontWeight: "700",
    marginBottom: 10,
  },
  resolutionBody: {
    color: "#4f5d75",
    lineHeight: 22,
  },
  detailGrid: {
    gap: 12,
    marginTop: 14,
  },
  detailCard: {
    backgroundColor: "#f9f4ec",
    borderRadius: 18,
    padding: 16,
  },
  detailLabel: {
```

```

        color: "#7b8794",
        fontSize: 12,
        fontWeight: "700",
        letterSpacing: 0.8,
        textTransform: "uppercase",
    },
    detailValue: {
        color: "#14213d",
        fontSize: 16,
        fontWeight: "700",
        marginTop: 8,
    },
    image: {
        width: "100%",
        height: 220,
        borderRadius: 24,
    },
},
});

```

Mobile Login Screen

File: mobile_app/src/screens/LoginScreen.js

```

import React, { useContext, useState } from "react";
import { Alert, SafeAreaView, StyleSheet, Text, TextInput, TouchableOpacity, View
} from "react-native";

import { AuthContext } from "../context/AuthContext";
import { loginUser } from "../services/authService";

export default function LoginScreen({ navigation }) {
    const { setUser } = useContext(AuthContext);
    const [email, setEmail] = useState("");
    const [password, setPassword] = useState("");

    const handleLogin = async () => {
        if (!email || !password) {
            Alert.alert("Missing details", "Please enter both email and password.");
            return;
        }

        try {
            const res = await loginUser({ email, password });
            setUser(res.user);
        } catch (err) {
            Alert.alert("Error", err?.response?.data?.message || "Login failed");
        }
    };

    return (

```

```

<SafeAreaView style={styles.safeArea}>
  <View style={styles.container}>
    <View style={styles.hero}>
      <Text style={styles.eyebrow}>UrbanEye</Text>
      <Text style={styles.title}>Sign in to your civic dashboard</Text>
    </View>

    <View style={styles.form}>
      <TextInput
        placeholder="Email"
        placeholderTextColor="#8d99ae"
        style={styles.input}
        autoCapitalize="none"
        value={email}
        onChangeText={setEmail}
      />
      <TextInput
        placeholder="Password"
        placeholderTextColor="#8d99ae"
        secureTextEntry
        style={styles.input}
        value={password}
        onChangeText={setPassword}
      />

      <TouchableOpacity style={styles.primaryButton} onPress={handleLogin}>
        <Text style={styles.primaryButtonText}>Login</Text>
      </TouchableOpacity>

      <TouchableOpacity onPress={() => navigation.navigate("Register")}>
        <Text style={styles.secondaryText}>Don't have an account?
Register</Text>
      </TouchableOpacity>
    </View>
  </View>
</SafeAreaView>
);
}

const styles = StyleSheet.create({
  safeArea: {
    flex: 1,
    backgroundColor: "#f4efe6",
  },
  container: {
    flex: 1,
    padding: 20,
    justifyContent: "center",
    gap: 18,
  },
  hero: {
    backgroundColor: "#14213d",

```

```
    borderRadius: 28,
    padding: 24,
  },
  eyebrow: {
    color: "#ef8354",
    fontSize: 12,
    fontWeight: "700",
    textTransform: "uppercase",
    letterSpacing: 2,
  },
  title: {
    color: "#fff",
    marginTop: 10,
    fontSize: 30,
    fontWeight: "800",
  },
  form: {
    backgroundColor: "#fff",
    borderRadius: 28,
    padding: 20,
    gap: 12,
  },
  input: {
    borderWidth: 1,
    borderColor: "#e5e7eb",
    padding: 14,
    borderRadius: 16,
    color: "#14213d",
  },
  primaryButton: {
    backgroundColor: "#ef8354",
    borderRadius: 999,
    paddingVertical: 14,
    alignItems: "center",
    marginTop: 4,
  },
  primaryButtonText: {
    color: "#fff",
    fontWeight: "700",
    fontSize: 16,
  },
  secondaryText: {
    color: "#4f5d75",
    textAlign: "center",
    marginTop: 6,
  },
},
});
```

Mobile Register Screen

File: mobile_app/src/screens/RegisterScreen.js

```

import React, { useContext, useState } from "react";
import { Alert, SafeAreaView, StyleSheet, Text, TextInput, TouchableOpacity, View
} from "react-native";

import { AuthContext } from "../context/AuthContext";
import { registerUser } from "../services/authService";

export default function RegisterScreen({ navigation }) {
  const { setUser } = useContext(AuthContext);
  const [name, setName] = useState("");
  const [email, setEmail] = useState("");
  const [password, setPassword] = useState("");

  const handleRegister = async () => {
    if (!name || !email || !password) {
      Alert.alert("Missing details", "Please fill in name, email, and password.");
      return;
    }

    try {
      const res = await registerUser({ name, email, password });
      setUser(res.user);
      Alert.alert("Success", "Account created");
    } catch (err) {
      Alert.alert("Error", err?.response?.data?.message || "Registration failed");
    }
  };

  return (
    <SafeAreaView style={styles.safeArea}>
      <View style={styles.container}>
        <View style={styles.hero}>
          <Text style={styles.eyebrow}>Create account</Text>
          <Text style={styles.title}>Join the city issue response network</Text>
        </View>

        <View style={styles.form}>
          <TextInput
            placeholder="Full name"
            placeholderTextColor="#8d99ae"
            style={styles.input}
            value={name}
            onChangeText={setName}
          />
          <TextInput
            placeholder="Email"
            placeholderTextColor="#8d99ae"
            style={styles.input}
            autoCapitalize="none"
            value={email}
            onChangeText={setEmail}
          />
        </View>
      </View>
    </SafeAreaView>
  );
}

```

```

        />
        <TextInput
          placeholder="Password"
          placeholderTextColor="#8d99ae"
          secureTextEntry
          style={styles.input}
          value={password}
          onChangeText={setPassword}
        />

        <TouchableOpacity style={styles.primaryButton} onPress={handleRegister}>
          <Text style={styles.primaryButtonText}>Register</Text>
        </TouchableOpacity>

        <TouchableOpacity onPress={() => navigation.navigate("Login")}>
          <Text style={styles.secondaryText}>Already have an account?
Login</Text>
        </TouchableOpacity>
      </View>
    </View>
  </SafeAreaView>
);
}

const styles = StyleSheet.create({
  safeArea: {
    flex: 1,
    backgroundColor: "#f4efe6",
  },
  container: {
    flex: 1,
    padding: 20,
    justifyContent: "center",
    gap: 18,
  },
  hero: {
    backgroundColor: "#14213d",
    borderRadius: 28,
    padding: 24,
  },
  eyebrow: {
    color: "#ef8354",
    fontSize: 12,
    fontWeight: "700",
    textTransform: "uppercase",
    letterSpacing: 2,
  },
  title: {
    color: "#fff",
    marginTop: 10,
    fontSize: 30,
    fontWeight: "800",
  },

```

```

    },
    form: {
      backgroundColor: "#fff",
      borderRadius: 28,
      padding: 20,
      gap: 12,
    },
    input: {
      borderWidth: 1,
      borderColor: "#e5e7eb",
      padding: 14,
      borderRadius: 16,
      color: "#14213d",
    },
    primaryButton: {
      backgroundColor: "#ef8354",
      borderRadius: 999,
      paddingVertical: 14,
      alignItems: "center",
      marginTop: 4,
    },
    primaryButtonText: {
      color: "#fff",
      fontWeight: "700",
      fontSize: 16,
    },
    secondaryText: {
      color: "#4f5d75",
      textAlign: "center",
      marginTop: 6,
    },
  },
});

```

Mobile Complaint Utilities

File: mobile_app/src/utlis/complaintUtils.js

```

const STATUS_META = {
  PENDING: {
    label: "Pending",
    shortLabel: "Not solved yet",
    helper: "Awaiting assignment",
    backgroundColor: "#fff1d9",
    borderColor: "#f3c677",
    textColor: "#9a5b00",
  },
  IN_PROGRESS: {
    label: "In Progress",
    shortLabel: "Being handled",
    helper: "Team is working on it",
  },
};

```

```

        backgroundColor: "#dbeafe",
        borderColor: "#93c5fd",
        textColor: "#1d4ed8",
    },
    RESOLVED: {
        label: "Solved",
        shortLabel: "Resolved",
        helper: "Complaint closed",
        backgroundColor: "#dcfce7",
        borderColor: "#86efac",
        textColor: "#166534",
    },
};

const PRIORITY_META = {
    LOW: {
        label: "Low",
        backgroundColor: "#eef2ff",
        textColor: "#4f46e5",
    },
    MEDIUM: {
        label: "Medium",
        backgroundColor: "#fef3c7",
        textColor: "#b45309",
    },
    HIGH: {
        label: "High",
        backgroundColor: "#ffe4e6",
        textColor: "#be123c",
    },
    CRITICAL: {
        label: "Critical",
        backgroundColor: "#fee2e2",
        textColor: "#b91c1c",
    },
};

export const DASHBOARD_FILTERS = [
    { key: "ALL", label: "All" },
    { key: "PENDING", label: "Pending" },
    { key: "IN_PROGRESS", label: "In Progress" },
    { key: "RESOLVED", label: "Solved" },
];

export function getStatusMeta(status = "PENDING") {
    return STATUS_META[status] || STATUS_META.PENDING;
}

export function getPriorityMeta(priority = "MEDIUM") {
    return PRIORITY_META[priority] || PRIORITY_META.MEDIUM;
}

```

```

export function getComplaintStats(complaints = []) {
  return complaints.reduce(
    (acc, complaint) => {
      acc.total += 1;

      if (complaint.status === "RESOLVED") {
        acc.resolved += 1;
      } else if (complaint.status === "IN_PROGRESS") {
        acc.inProgress += 1;
      } else {
        acc.pending += 1;
      }

      return acc;
    },
    {
      total: 0,
      pending: 0,
      inProgress: 0,
      resolved: 0,
    },
  );
}

export function filterComplaints(complaints = [], filter = "ALL") {
  if (filter === "ALL") {
    return complaints;
  }

  return complaints.filter((complaint) => complaint.status === filter);
}

export function getResolutionMessage(status = "PENDING") {
  if (status === "RESOLVED") {
    return "This complaint has been solved and marked closed.";
  }

  if (status === "IN_PROGRESS") {
    return "A response team is already working on this complaint.";
  }

  return "This complaint is still open and waiting for action.";
}

```